Network Security

From Zero to Expert

A step-by-step booklet with diagrams, worked examples
and Python labs for undergraduate students



Prof. Mahmoud Abdelaziz

Military Technical College, Cairo, Egypt

mkaremm@mtc.edu.eg | github.com/mkaremm80

Version 1 — 2026

How to Use This Booklet vii

1 Introduction to Network Security 1

1.1 Why network security? . . . 1
1.2 The CIA triad . . . 1
1.3 The basic words . . . 2
1.4 Who are the attackers? . . . 2
1.5 Types of attacks . . . 3
1.5.1 Passive and active attacks . . . 3
1.5.2 The attack life cycle (kill chain) . . . 3
1.6 Defense in depth . . . 3
1.7 Basic security principles . . . 4
1.8 Measuring risk . . . 4

2 Cryptography Basics 6

2.1 The model of encryption . . . 6
2.1.1 How strong is a key? . . . 6
2.2 Classical ciphers . . . 7
2.2.1 Caesar cipher . . . 7
2.2.2 Substitution and frequency analysis . . . 7
2.2.3 The one-time pad . . . 7
2.3 Symmetric encryption . . . 7
2.3.1 Block ciphers and AES . . . 7
2.3.2 Modes of operation . . . 8
2.3.3 Authenticated encryption . . . 8
2.4 Public-key cryptography . . . 9
2.4.1 Some number theory . . . 9
2.4.2 RSA . . . 9
2.4.3 Diffie–Hellman key exchange . . . 10
2.4.4 Elliptic curve cryptography (ECC) . . . 11
2.5 Hash functions . . . 11
2.6 Message authentication codes (MAC) . . . 12
2.7 Digital signatures . . . 12
2.8 Certificates and PKI . . . 13
2.9 Key management . . . 13

3 Authentication and Access Control **15**

3.1 Identification, authentication, authorization . . . 15
3.2 Authentication factors . . . 15
3.3 Passwords . . . 15
3.3.1 Why passwords fail . . . 15
3.3.2 Password strength (entropy) . . . 16
3.3.3 Storing passwords on the server . . . 16
3.4 One-time passwords . . . 17
3.5 Challenge–response . . . 18
3.6 Kerberos and single sign-on . . . 18
3.7 Access control models . . . 18

4 Network Fundamentals and Attacks **20**

4.1 A short review of TCP/IP . . . 20
4.1.1 The TCP three-way handshake . . . 20
4.1.2 Reading a packet . . . 21
4.2 Sniffing . . . 21
4.3 ARP poisoning . . . 21
4.4 IP spoofing . . . 22
4.5 DNS attacks . . . 22
4.6 Reconnaissance and scanning . . . 22
4.7 Session hijacking . . . 23
4.8 Denial of service . . . 23
4.8.1 SYN flood . . . 23
4.8.2 Reflection and amplification . . . 23
4.8.3 Defending against DDoS . . . 23

5 Network Security Protocols **25**

5.1 Where to put security? . . . 25
5.2 TLS: Transport Layer Security . . . 25
5.2.1 The TLS 1.3 handshake . . . 26
5.2.2 Cipher suites . . . 26
5.3 SSH: Secure Shell . . . 27
5.4 IPsec . . . 27
5.4.1 AH and ESP . . . 27
5.4.2 Transport mode and tunnel mode . . . 28
5.4.3 Security associations and IKE . . . 28
5.5 Comparing TLS, SSH and IPsec . . . 28

6 Firewalls and Intrusion Detection 30

6.1 What is a firewall? . . . 30
6.2 Types of firewalls . . . 30
6.2.1 Packet filters (stateless) . . . 30
6.2.2 Stateful firewalls . . . 30
6.2.3 Application proxies . . . 30
6.2.4 Next-generation firewalls (NGFW) . . . 31
6.3 Writing firewall rules . . . 31
6.4 Network design: DMZ and segmentation . . . 32
6.5 Intrusion detection and prevention . . . 32
6.5.1 Detection methods . . . 32
6.5.2 Measuring an IDS . . . 33
6.6 Honeypots . . . 34

7 Virtual Private Networks (VPNs) 35

7.1 What is a VPN? . . . 35
7.2 Types of VPN . . . 35
7.2.1 Remote-access VPN . . . 35
7.2.2 Site-to-site VPN . . . 36
7.2.3 Full tunnel and split tunnel . . . 36
7.3 VPN technologies . . . 36
7.3.1 IPsec with IKEv2 . . . 36
7.3.2 TLS-based VPNs (SSL VPN, OpenVPN) . . . 36
7.3.3 WireGuard . . . 36
7.3.4 Comparison . . . 37
7.4 Tunnel overhead and MTU . . . 37
7.5 Lab: a toy ESP tunnel . . . 38

8 Wireless and Mobile Security 40

8.1 Why wireless is different . . . 40
8.2 The history of Wi-Fi security . . . 40
8.2.1 WEP: a lesson in bad design . . . 40
8.2.2 WPA and WPA2 . . . 41
8.3 How WPA2 makes its keys . . . 41
8.3.1 From passphrase to PMK . . . 41
8.3.2 The 4-way handshake . . . 41
8.4 WPA3 . . . 42
8.5 Common wireless attacks . . . 43
8.6 Securing a Wi-Fi network: checklist . . . 43

8.7 Bluetooth and mobile security . . . 43
8.7.1 Bluetooth . . . 43
8.7.2 Mobile phones . . . 43
8.7.3 Cellular networks . . . 44

9 Malware and Application Attacks 45
9.1 Malware types . . . 45
9.1.1 How a worm spreads . . . 45
9.2 Ransomware . . . 46
9.3 Advanced persistent threats (APT) . . . 46
9.4 Detecting and stopping malware . . . 46
9.5 Social engineering . . . 47
9.6 Web application attacks . . . 47
9.6.1 SQL injection . . . 47
9.6.2 Cross-site scripting (XSS) . . . 48
9.6.3 Cross-site request forgery (CSRF) . . . 48
9.6.4 Other important web bugs . . . 49
9.6.5 Buffer overflow (a classic) . . . 49

10 Security Management and Monitoring 51
10.1 Security is a cycle . . . 51
10.2 Security policies . . . 51
10.3 Standards and frameworks . . . 52
10.4 Risk assessment . . . 52
10.4.1 Qualitative . . . 52
10.4.2 Quantitative . . . 52
10.5 Logging and monitoring . . . 53
10.5.1 SIEM and SOC . . . 53
10.6 Vulnerability management . . . 54
10.6.1 Penetration testing . . . 54
10.7 Incident response . . . 54
10.8 Business continuity and disaster recovery . . . 55

11 Emerging Topics 57
11.1 Zero trust . . . 57
11.2 Cloud security . . . 58
11.3 IoT and industrial networks . . . 58
11.3.1 Internet of Things . . . 58
11.3.2 Operational technology (OT / ICS / SCADA) . . . 59

11.4 Quantum computing and post-quantum cryptography . . . 59
11.4.1 The threat . . . 59
11.4.2 The new standards . . . 59
11.4.3 Hash-based signatures: Lamport . . . 60
11.5 AI and security . . . 61
11.6 Your path to expert . . . 61

12 Capstone: Securing a Campus Network 62
12.1 The scenario . . . 62
12.2 Step 1: Identify assets and risks . . . 62
12.3 Step 2: Design the new network . . . 62
12.4 Step 3: Identity and access . . . 63
12.5 Step 4: Detect . . . 63
12.6 Step 5: Respond and recover . . . 63
12.7 Step 6: Prepare for the future . . . 63
12.8 Final self-test . . . 64

A Building Your Practice Lab 65
A.1 Software you need . . . 65
A.2 A safe home lab design . . . 65
A.3 Suggested lab sessions . . . 65

B Answers to Selected Exercises 67

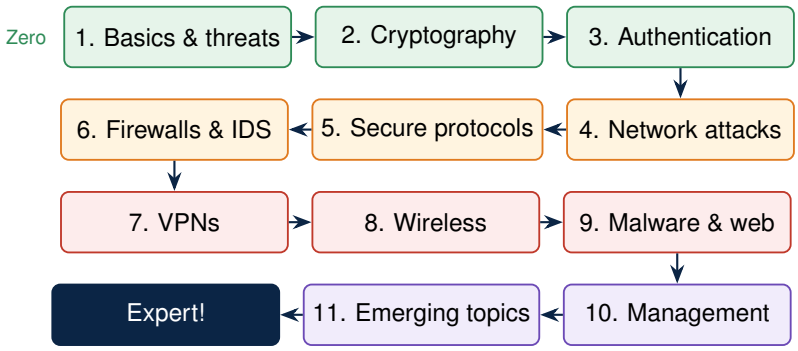
C Quick Reference 69
C.1 Common ports . . . 69
C.2 Which algorithm should I use? (2026) . . . 69
C.3 Top 10 actions for any organization . . . 69

D Glossary 70

E Further Reading 72

This booklet takes you from *zero* (you only know what a computer network is) to an *expert* view of how real networks are attacked and defended. Each chapter builds on the one before it, so please read them in order the first time.

The road map



The colors show four levels: **foundations** (Ch. 1–3), **the network** (Ch. 4–6), **special areas** (Ch. 7–9) and **expert view** (Ch. 10–11). Chapter 12 is a capstone project that joins everything, with a final self-test.

The boxes you will see

What you will learn

At the start of each chapter: the goals of the chapter.

Key idea

The most important ideas. If you only read one thing, read these.

Example

Worked examples with numbers, step by step.

Be careful

Common mistakes that cause real security problems.

Expert corner

Deeper material. You can skip it on the first reading.

Python labs

Every chapter has small Python programs. All of them were run before printing, and the output you see is the real output. You need Python 3.9 or newer and one extra package:

```
pip install cryptography
```

All labs work on your own computer. They do not attack any real system.

Ethics first

Only test systems that you own or that you have **written permission** to test. Attacking other people's networks is a crime in Egypt and in almost every country, even "just for learning". In this booklet we learn attacks only so that we can defend against them.

Exercise levels

[B] = beginner, **[I]** = intermediate, **[E]** = expert. Short answers to selected exercises are in Appendix B.

CHAPTER 1

Introduction to Network Security

What you will learn

- Understand what “security” means using the CIA triad.
- Know the words: asset, threat, vulnerability, attack, risk and control.
- Know the main types of attackers and attacks.
- Learn the idea of defense in depth and compute a simple risk score.

1.1 Why network security?

Today almost everything is connected: banks, hospitals, power stations, universities and military systems. A computer network lets information move fast, but it also lets *attackers* reach a system from far away, without touching it. One weak laptop can open the door to a whole organization.

Network security is the set of ideas, tools and rules that protect data **while it is stored, while it moves over the network, and while it is used**, and that keep the network itself working.

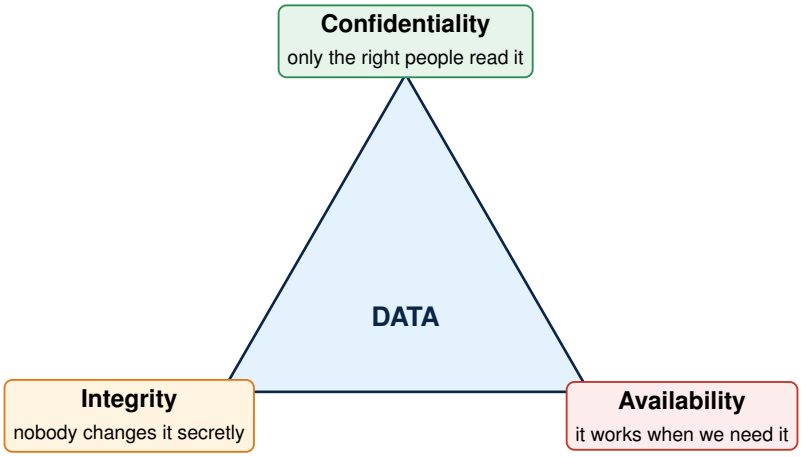
Real cases

- **WannaCry (2017):** ransomware spread by itself over networks through an old Windows file-sharing bug. It locked hundreds of thousands of computers, including hospital systems.
- **Mirai (2016):** a botnet of cheap cameras and home routers that still had their factory passwords. It produced one of the biggest DDoS attacks of its time and took many famous web sites offline.
- **SolarWinds (2020):** attackers hid malicious code inside a trusted software update, so thousands of companies installed it themselves.

Each case used a different weakness: an old bug, a weak password, and trust in a supplier.

1.2 The CIA triad

Security people describe their goals with three letters: **C**onfidentiality, **I**ntegrity and **A**vailability.



Goal	Meaning	Attack against it / tool to protect it
Confidentiality	Data is read only by people who are allowed.	Sniffing, data theft / encryption, access control
Integrity	Data is not changed without being detected.	Changing a bank transfer / hashes, MACs, signatures
Availability	The service works when users need it.	DoS, ransomware / backups, redundancy, filtering

Two more goals are often added: **authentication** (we know *who* we talk to) and **non-repudiation** (a person cannot later deny that they sent a message). Together these five goals are the base of the whole booklet.

1.3 The basic words

Asset Anything of value: a database, a server, a password, the reputation of the college.

Vulnerability A weakness: a software bug, a weak password, an open port, an untrained user.

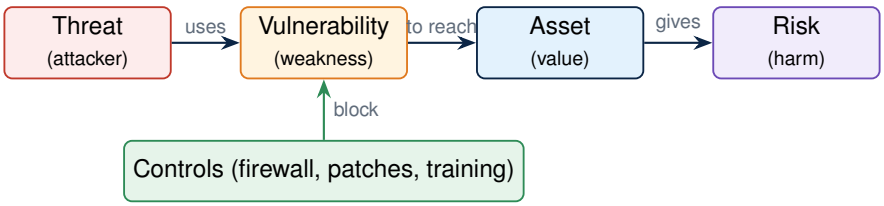
Threat Something that *could* use a vulnerability to cause harm (a hacker, malware, even a flood).

Attack The real action that uses the vulnerability.

Exploit The code or method that uses a specific vulnerability.

Risk How likely the harm is, multiplied by how bad it would be.

Control A protection that lowers risk (a firewall, a policy, training).



Key idea

There is **no risk** if there is no vulnerability, or no threat, or no asset of value. Security work is about finding the vulnerabilities in valuable assets and closing them before a threat finds them.

1.4 Who are the attackers?

Attacker	Goal	Skill
Script kiddie	Fun, showing off. Uses ready-made tools.	Low
Cyber criminal	Money: ransomware, card theft, fraud.	Medium–high
Hacktivist	A political or social message.	Medium
Insider	Revenge or money. Already inside the network.	Varies
Nation state (APT)	Spying, sabotage. Long, hidden campaigns.	Very high

Be careful

Do not forget the **insider**. Many studies show that a large part of incidents involve people inside the organization, often by mistake (sending a file to the wrong person, clicking a phishing link).

1.5 Types of attacks

1.5.1 Passive and active attacks

- **Passive attacks** only *listen*: eavesdropping and traffic analysis. They are hard to detect, so we *prevent* them with encryption.
- **Active attacks** *change* something: modification, fake messages (masquerade), replay of old messages and denial of service. They are easier to detect, so we *detect and recover*.



In security books, *Alice* and *Bob* are the good users, *Eve* is an eavesdropper and *Mallory* is an active (malicious) attacker.

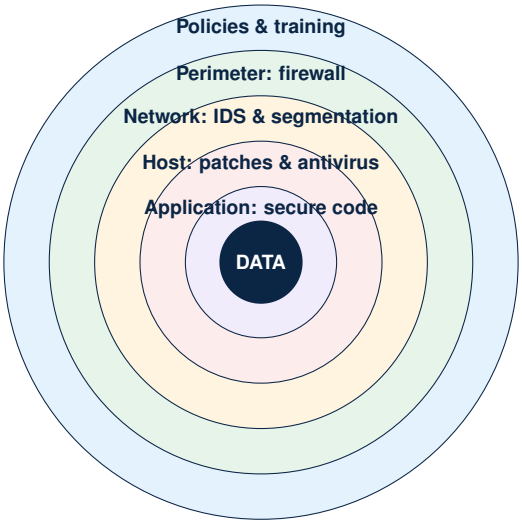
1.5.2 The attack life cycle (kill chain)

Real attacks follow steps. If we break any step, we stop the attack.



1.6 Defense in depth

A castle has a river, a wall, guards and a locked room. If one layer fails, the next one still works. Networks use the same idea.



Controls are often put in three groups:

- **Technical:** firewalls, encryption, antivirus.
- **Administrative:** policies, training, background checks.

- **Physical:** locks, guards, cameras.

And by *what they do*: **prevent** (firewall), **detect** (IDS), **correct** (backup restore).

1.7 Basic security principles

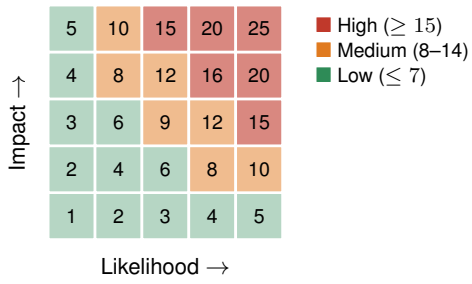
- **Least privilege:** give each user only the rights they need, no more.
- **Separation of duties:** one person should not be able to do a critical task alone.
- **Fail secure:** when something breaks, it should block access, not open it.
- **Keep it simple:** complex systems hide more bugs.
- **No security by obscurity:** a system must stay safe even if the attacker knows how it works. Only the *key* is secret (Kerckhoffs's principle).

1.8 Measuring risk

A simple way to rank risks is

$$\text{Risk} = \text{Likelihood} \times \text{Impact},$$

where each is given a score from 1 (very low) to 5 (very high). The highest risks are treated first.



After you find a risk you can: **reduce** it (add a control), **transfer** it (insurance), **avoid** it (stop the activity) or **accept** it (if it is small).

Lab — Risk ranking `(c01_risk.py)`

```python
# Chapter 1: simple risk score = likelihood x impact
assets = [
    # name,            likelihood(1-5), impact(1-5)
    ("Web server",      4, 4),
    ("Student database", 3, 5),
    ("Office Wi-Fi",    4, 2),
    ("Backup server",   2, 5),
]

def level(score):
    if score >= 15:
        return "HIGH"
    if score >= 8:
        return "MEDIUM"
    return "LOW"

print(f"{'Asset':18}{'L':>3}{'I':>3}{'Risk':>6}  Level")
for name, l, i in sorted(assets, key=lambda a: -a[1] * a[2]):
    r = l * i
    print(f"{name:18}{l:>3}{i:>3}{r:>6}  {level(r)}")
```

Output:

```
Asset              L  I  Risk  Level
Web server         4  4    16  HIGH
Student database   3  5    15  HIGH
Backup server      2  5    10  MEDIUM
Office Wi-Fi       4  2     8  MEDIUM
```

Chapter summary

- Security goals: confidentiality, integrity, availability (+ authentication, non-repudiation).
- Risk exists when a threat can use a vulnerability in a valuable asset.
- Attacks are passive (listen) or active (change); they follow a kill chain.
- Use many layers of defense and the principle of least privilege.

Exercises

1. **[B]** For each case, say which CIA goal is broken: (a) a student reads the exam before the exam day; (b) a virus changes the marks in a file; (c) the college web site is down for two days.
2. **[B]** Give one example of a technical, an administrative and a physical control in your college.
3. **[I]** Explain why a passive attack is “prevented” but an active attack is “detected”.
4. **[I]** Add three assets from your own home network to the lab program and rank them.
5. **[E]** Pick the WannaCry case. Map each of its steps to the kill chain and say which control could stop each step.

CHAPTER 2

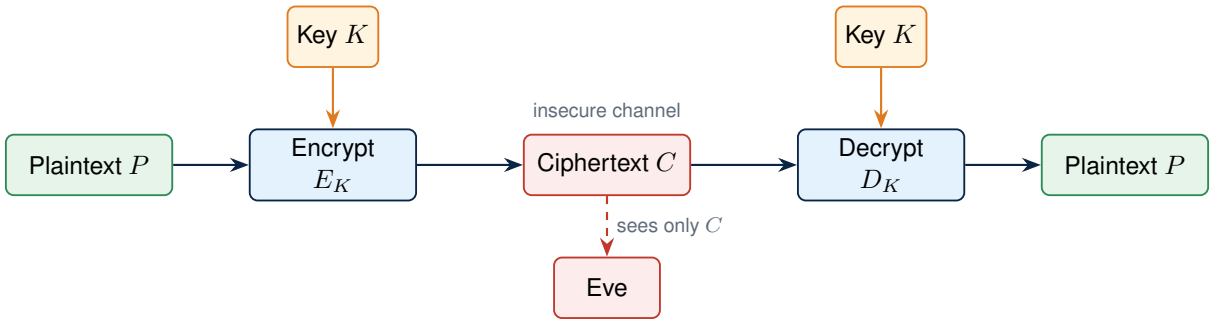
Cryptography Basics

What you will learn

- Understand the model of encryption: plaintext, key, ciphertext.
- Learn symmetric ciphers (AES) and their modes of operation.
- Learn public-key cryptography (RSA, Diffie–Hellman, ECC) with small numbers.
- Understand hashes, MACs, digital signatures, certificates and PKI.
- Build each tool in Python and see it work.

Cryptography is the *engine* of network security. Almost every protocol in this booklet (TLS, IPsec, WPA, SSH) is a smart way to combine a few cryptographic building blocks. If you understand this chapter well, the rest will be easy.

2.1 The model of encryption



We write $C = E_K(P)$ and $P = D_K(C)$. Following Kerckhoffs's principle, the algorithms E and D are public; only the key K is secret.

2.1.1 How strong is a key?

If a key has n bits, there are 2^n possible keys. A brute-force attacker tries all of them; on average he needs 2^{n-1} tries.

Key size and time

Assume a very strong attacker who can test 10^{12} keys per second.

Key size	Number of keys	Average time to break
40 bits	1.1×10^{12}	about 0.5 seconds
56 bits (DES)	7.2×10^{16}	about 10 hours
128 bits (AES)	3.4×10^{38}	about 5×10^{18} years
256 bits (AES)	1.2×10^{77}	far longer than the age of the universe

This is why DES is dead and AES-128 is still safe.

2.2 Classical ciphers

2.2.1 Caesar cipher

Each letter is moved k places in the alphabet: $C = (P + k) \bmod 26$. With $k = 3$, ATTACK becomes DWWDFN. There are only 25 useful keys, so it is broken in a second.

Lab — Caesar cipher and brute force `(c02_caesar.py)`

```python
# Chapter 2: Caesar cipher and a brute-force attack
def caesar(text, key):
    out = ""
    for ch in text.upper():
        if ch.isalpha():
            out += chr((ord(ch) - 65 + key) % 26 + 65)
        else:
            out += ch
    return out

c = caesar("ATTACK AT DAWN", 3)
print("Cipher text:", c)
print("Decrypted  :", caesar(c, -3))

# Only 26 keys -> try them all
for k in range(1, 4):
    print(k, caesar(c, -k))
```

Output:

```
Cipher text: DWWDFN DW GDZQ
Decrypted  : ATTACK AT DAWN
1 CVVCEM CV FCYP
2 BUUBDL BU EBXO
3 ATTACK AT DAWN
```

2.2.2 Substitution and frequency analysis

If each letter is replaced by another random letter, there are $26! \approx 4 \times 10^{26}$ keys. But this is still weak: in English, “E” is the most common letter (about 12%), then “T” and “A”. The attacker counts letters in the ciphertext and guesses. **Lesson:** a big key space is necessary, but it is not enough. A good cipher must also hide the statistics of the plaintext.

2.2.3 The one-time pad

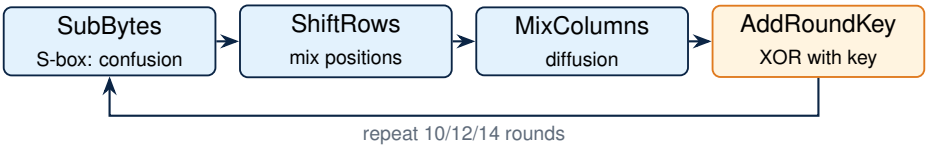
If the key is truly random, as long as the message, and used only once, then $C = P \oplus K$ is *perfectly* secure (Shannon, 1949). The problem is practical: how do we share a key as long as all our messages? Modern stream ciphers try to imitate the one-time pad with a short key.

2.3 Symmetric encryption

In symmetric (secret-key) cryptography, both sides use the **same** key. It is very fast and is used to encrypt almost all data on the Internet.

2.3.1 Block ciphers and AES

A block cipher encrypts a fixed block of bits. The **Advanced Encryption Standard (AES)** works on 128-bit blocks with keys of 128, 192 or 256 bits. It repeats a *round* 10, 12 or 14 times. Each round has four steps:



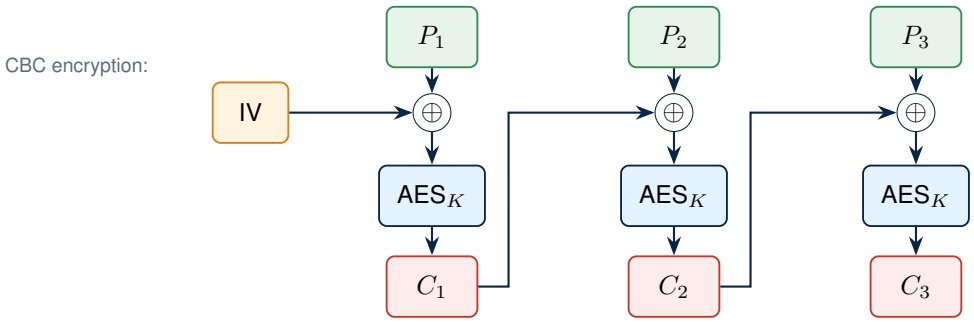
Confusion and diffusion (Shannon)

Confusion: the link between key and ciphertext must be complex (done by the S-box). **Diffusion:** changing one plaintext bit must change about half of the ciphertext bits (done by ShiftRows and MixColumns). This is called the *avalanche effect*.

2.3.2 Modes of operation

AES encrypts only 16 bytes. To encrypt a long message we need a *mode*.

Mode	How it works	Use today?
ECB	Each block encrypted alone. Same plaintext block ⇒ same ciphertext block.	Never
CBC	Each block is XORed with the previous ciphertext block first. Needs a random IV.	Old systems
CTR	Encrypts a counter and XORs it with the data (a stream cipher).	Yes, inside GCM
GCM	CTR encryption + a built-in authentication tag.	Recommended



The ECB penguin

If you encrypt a picture with ECB, you can still see the shape of the picture in the ciphertext, because equal blocks give equal outputs. Never use ECB. Also, in CTR and GCM **never reuse a nonce** with the same key: two messages with the same nonce leak $P_1 \oplus P_2$, and in GCM the attacker can even forge messages.

2.3.3 Authenticated encryption

Encryption alone gives confidentiality, *not* integrity. An attacker can flip bits in CTR ciphertext and the same bits flip in the plaintext. AES-GCM (and ChaCha20-Poly1305) solve this: they add a tag, and decryption fails if anything was changed. This is called AEAD (authenticated encryption with associated data).

Lab — AES-GCM detects tampering `(c02_aes_gcm.py)`

```
# Chapter 2: authenticated encryption with AES-GCM
# pip install cryptography
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

key = AESGCM.generate_key(bit_length=256)   # 32-byte secret key
aes = AESGCM(key)
```

```python
nonce = os.urandom(12)                      # never reuse with same key!
header = b"to:server1"                      # authenticated, not encrypted

ct = aes.encrypt(nonce, b"Exam marks: 95, 88, 72", header)
print("Cipher text (hex):", ct.hex()[:48], "...")
print("Plain text       :", aes.decrypt(nonce, ct, header))

# An attacker flips one bit
bad = bytearray(ct); bad[0] ^= 1
try:
    aes.decrypt(nonce, bytes(bad), header)
except Exception as e:
    print("Tampering detected:", type(e).__name__)
```

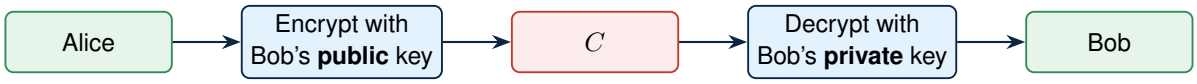
Output:

```
Cipher text (hex): ef2d0549c1229a8f08bf7d227d6dc6bd83390f38c07ddcb1 ...
Plain text       : b'Exam marks: 95, 88, 72'
Tampering detected: InvalidTag
```

2.4 Public-key cryptography

Symmetric crypto has one big problem: *how do Alice and Bob agree on a key* if they never met? With N users, each pair needs its own key: $N(N-1)/2$ keys. For 1000 users that is almost half a million keys!

Public-key (asymmetric) cryptography gives each user **two keys**: a *public key* that anyone can know, and a *private key* that only the owner knows.



2.4.1 Some number theory

We need only three ideas:

- **Modular arithmetic:** $a \bmod n$ is the remainder after dividing by n. Example: $17 \bmod 5 = 2$.
- **Euler's function:** for $n = pq$ with primes p, q: $\varphi(n) = (p-1)(q-1)$.
- **Euler's theorem:** if $\gcd(m, n) = 1$ then $m^{\varphi(n)} \equiv 1 \pmod{n}$.

Computing $a^b \bmod n$ is fast even for huge numbers (square-and-multiply). But the reverse problems, *factoring* n and finding *discrete logarithms*, are very hard. Public-key crypto is built on these "one-way" problems.

2.4.2 RSA

RSA step by step

Key generation

1. Choose two primes: $p = 61$, $q = 53$.
2. $n = pq = 3233$ and $\varphi(n) = 60 \times 52 = 3120$.
3. Choose $e = 17$ with $\gcd(17, 3120) = 1$.
4. Find d with $ed \equiv 1 \pmod{3120}$: $d = 2753$ (check: $17 \times 2753 = 46801 = 15 \times 3120 + 1$).

Public key $= (e, n) = (17, 3233)$. Private key $= d = 2753$.
Encrypt $m = 65$: $c = 65^{17} \bmod 3233 = 2790$.
Decrypt: $m = 2790^{2753} \bmod 3233 = 65$. ✓

Why it works: $c^d = m^{ed} = m^{1+k\varphi(n)} = m \cdot (m^{\varphi(n)})^k \equiv m \pmod{n}$.
Why it is safe: to find d, the attacker needs $\varphi(n)$, which needs p and q. Factoring a 2048-bit n is far beyond today's computers.

Lab — Textbook RSA `(c02_rsa_toy.py)`

```python
# Chapter 2: textbook RSA with small numbers (for learning only!)
p, q = 61, 53
n = p * q                  # 3233
phi = (p - 1) * (q - 1)    # 3120
e = 17                     # public exponent, gcd(e, phi) = 1
d = pow(e, -1, phi)        # private exponent = 2753

m = 65                     # message (must be < n)
c = pow(m, e, n)           # encrypt with public key (e, n)
m2 = pow(c, d, n)          # decrypt with private key d
print(f"n={n} phi={phi} d={d}")
print(f"m={m} -> c={c} -> m={m2}")

# Signature: sign with d, verify with e
s = pow(m, d, n)
print("signature", s, "valid:", pow(s, e, n) == m)
```

Output:

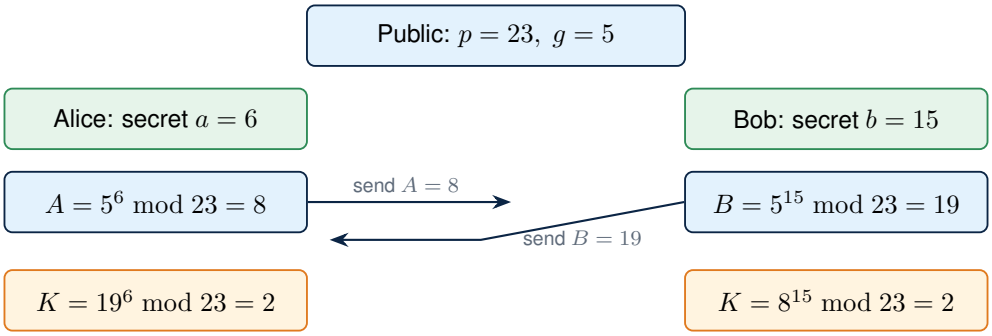
```
n=3233 phi=3120 d=2753
m=65 -> c=2790 -> m=65
signature 588 valid: True
```

Textbook RSA is not safe

Real RSA must use padding (OAEP for encryption, PSS for signatures). Without padding, the same message always gives the same ciphertext, and an attacker can multiply ciphertexts to get related messages. Use a library; never write your own RSA for real data.

2.4.3 Diffie–Hellman key exchange

Diffie and Hellman (1976) showed how two people can agree on a secret over a public channel.



Eve sees p, g, A, B but to get K she must solve the discrete logarithm problem.

Lab — Diffie–Hellman `(c02_dh_toy.py)`

```
# Chapter 2: Diffie-Hellman key exchange with small numbers
p, g = 23, 5              # public values
a, b = 6, 15              # secrets of Alice and Bob
A = pow(g, a, p)          # Alice sends A
B = pow(g, b, p)          # Bob sends B
print("Public A, B:", A, B)
print("Alice key:", pow(B, a, p))
print("Bob key  :", pow(A, b, p))
```

Output:

```
Public A, B: 8 19
Alice key: 2
Bob key  : 2
```

Man-in-the-middle

Plain Diffie–Hellman does not check *who* is on the other side. Mallory can do one exchange with Alice and another with Bob, and read everything. The fix is to **sign** the DH values (this is what TLS does, Chapter 5).

2.4.4 Elliptic curve cryptography (ECC)

ECC does the same job as RSA and DH, but on points of a curve $y^2 = x^3 + ax + b$ over a finite field. Its hard problem is harder, so keys are much shorter:

Security level	RSA / DH key	ECC key
80 bits	1024	160
128 bits	3072	256
256 bits	15360	512

Today ECC (X25519 for key exchange, Ed25519 and ECDSA for signatures) is the normal choice in TLS, SSH and mobile apps.

2.5 Hash functions

A hash function H maps any message to a short fixed-size *digest*. SHA-256 gives 256 bits. A good hash must have:

- **Pre-image resistance:** given h, it is hard to find m with $H(m) = h$.
- **Second pre-image resistance:** given m_1, hard to find $m_2 \neq m_1$ with the same hash.
- **Collision resistance:** hard to find *any* two messages with the same hash.

The birthday paradox

In a class of only 23 students, there is a 50% chance that two share a birthday. In the same way, for an n-bit hash, a collision is found after about $2^{n/2}$ tries, not 2^n. So a 256-bit hash gives only 128-bit collision security. MD5 (128 bits) and SHA-1 (160 bits) are broken; real collisions were found. Use SHA-256, SHA-3 or BLAKE2.

2.6 Message authentication codes (MAC)

A hash alone does not prove who sent the message: Mallory can change the message and compute a new hash. A MAC uses a *secret key*: tag $=$ MAC$_K(m)$. Only someone with K can make a valid tag. The common one is HMAC:

$$\text{HMAC}_K(m) = H\big((K \oplus \text{opad}) \,\|\, H((K \oplus \text{ipad}) \,\|\, m)\big).$$

Lab — Hash and HMAC (c02_hash_hmac.py)

```python
# Chapter 2: hash functions and HMAC
import hashlib, hmac

for msg in [b"hello", b"hellp"]:
    print(msg, hashlib.sha256(msg).hexdigest()[:32])

key = b"shared-secret"
tag = hmac.new(key, b"transfer 100 EGP", hashlib.sha256).hexdigest()
print("HMAC tag:", tag[:32])

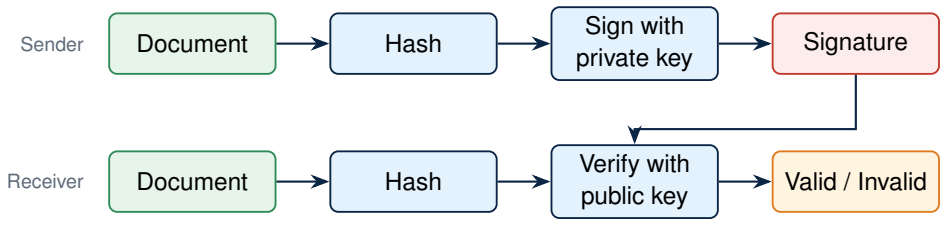
# Receiver checks the tag (constant-time compare)
check = hmac.new(key, b"transfer 900 EGP", hashlib.sha256).hexdigest()
print("Changed message accepted?", hmac.compare_digest(tag, check))
```

Output:

```
b'hello' 2cf24dba5fb0a30e26e83b2ac5b9e29e
b'hellp' fdd7585e08c4e2afd71dcabdb4636c89
HMAC tag: 1aa65320d64895e0c3f6922f1cb1f032
Changed message accepted? False
```

2.7 Digital signatures

A signature is like a MAC, but uses the **private key** to sign and the **public key** to verify. So anyone can check it, and only the owner could have made it. This gives integrity, authentication **and** non-repudiation.



Lab — Ed25519 signature (c02_sign.py)

```python
# Chapter 2: digital signature with Ed25519
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
from cryptography.exceptions import InvalidSignature

priv = Ed25519PrivateKey.generate()
pub = priv.public_key()
```

```python
doc = b"Final grades approved by Dr. Ahmed"
sig = priv.sign(doc)
print("Signature length:", len(sig), "bytes")

pub.verify(sig, doc)            # no error -> valid
print("Original document: valid")
try:
    pub.verify(sig, doc + b"!")
except InvalidSignature:
    print("Changed document: INVALID")
```

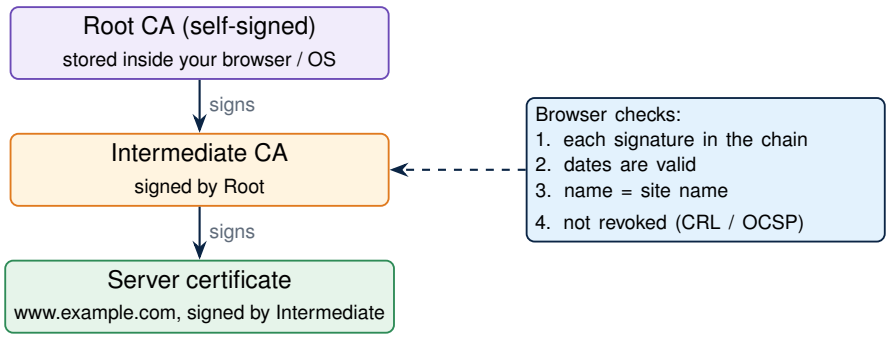
Output:

```
Signature length: 64 bytes
Original document: valid
Changed document: INVALID
```

Tool	Conf.	Integrity	Non-rep.	Key
Hash	–	only against accidents	–	none
MAC	–	✓	–	shared secret
Signature	–	✓	✓	private / public pair
AEAD (AES-GCM)	✓	✓	–	shared secret

2.8 Certificates and PKI

How does Alice know that a public key really belongs to Bob and not to Mallory? A trusted third party, the **Certificate Authority (CA)**, signs a statement: "this public key belongs to `www.mtc.edu.eg`". This signed statement is an **X.509 certificate**. It contains the owner name, the public key, the validity dates, the issuer name and the CA signature.



The whole system of CAs, certificates, registration and revocation is called the **Public Key Infrastructure (PKI)**.

2.9 Key management

Most real failures are not in the math but in the keys:

- Generate keys with a strong random generator (`os.urandom`, `secrets`), never `random`.
- Keep private keys in protected storage: hardware security modules (HSM), smart cards, TPM chips.
- Change (rotate) keys regularly and revoke keys that may be stolen.
- Use **hybrid encryption**: public-key crypto to share a short session key, then AES for the data. Every real protocol works like this.

Chapter summary

- Symmetric (AES): fast, same key on both sides; use GCM mode and never reuse a nonce.
- Public-key (RSA, DH, ECC): solves key sharing; built on hard math problems.
- Hashes check integrity; MACs add a secret key; signatures add non-repudiation.
- Certificates bind a public key to a name; CAs form the PKI.

Exercises

1. **[B]** Decrypt the Caesar ciphertext `KHOOR` (key 3).
2. **[B]** Why is ECB mode dangerous? Draw a small example.
3. **[I]** RSA with $p = 11$, $q = 17$, $e = 7$. Find n, $\varphi(n)$, d, then encrypt $m = 88$.
4. **[I]** DH with $p = 23$, $g = 5$, $a = 4$, $b = 3$. Find A, B and the shared key.
5. **[I]** Change one letter in the signed document of the Ed25519 lab. What happens and why?
6. **[E]** Explain why reusing a nonce in AES-CTR reveals $P_1 \oplus P_2$. Write a Python program that shows it.
7. **[E]** How many people are needed in a group so that the chance of a shared birthday is above 50%? Write code to compute it.

CHAPTER 3

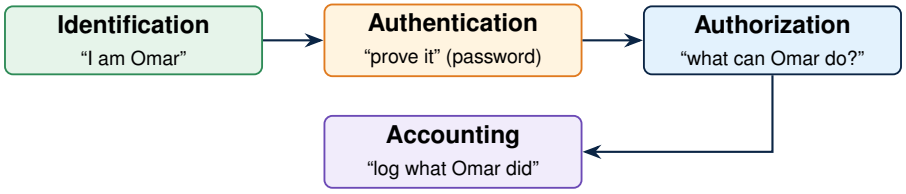
Authentication and Access Control

What you will learn

- Know the three factors of authentication and why MFA is strong.
- Store passwords safely with salt and slow hashes.
- Build one-time password codes (TOTP) in Python.
- Understand challenge–response, Kerberos and single sign-on.
- Compare access control models: DAC, MAC, RBAC and ABAC.

3.1 Identification, authentication, authorization

These three words are often mixed up:



Together they are called **AAA** (Authentication, Authorization, Accounting). In networks, AAA servers such as RADIUS and TACACS+ do this job for routers, Wi-Fi and VPNs.

3.2 Authentication factors

Factor	Examples	Weakness
Something you *know*	password, PIN, security question	guessed, phished, reused
Something you *have*	phone app, smart card, hardware key	lost or stolen
Something you *are*	fingerprint, face, iris	cannot be changed if leaked; false matches

Multi-factor authentication (MFA) uses factors from *different* groups. A password plus a PIN is still one factor (both are “know”). A password plus a code from your phone is two factors. To break in, the attacker must steal both.

Key idea

MFA blocks the large majority of automated account attacks. If you can only add one control to a system, MFA is often the best choice.

3.3 Passwords

3.3.1 Why passwords fail

- **Guessing / brute force:** trying many passwords online on the login page.

- **Dictionary attack:** trying common words and leaked passwords.
- **Credential stuffing:** using passwords leaked from one site on another site (people reuse passwords).
- **Phishing:** the user types the password into a fake site.
- **Offline cracking:** the attacker steals the password *database* and tests guesses on his own machine at billions per second.

3.3.2 Password strength (entropy)

If a password has L characters chosen at random from an alphabet of N symbols, its strength is

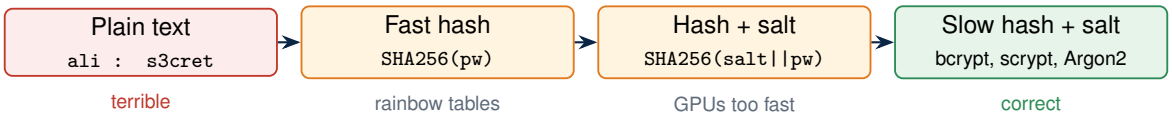
$$\text{Entropy} = L \log_2 N \text{ bits.}$$

Example

- 8 random lowercase letters: $8 \log_2 26 = 37.6$ bits.
- 8 random characters from 94 printable symbols: $8 \log_2 94 = 52.4$ bits.
- 4 random words from a list of 7776 words (passphrase): $4 \log_2 7776 = 51.7$ bits, and much easier to remember.
- 6 random words: 77.5 bits.

Note: this is only true if the choice is *random*. "Password1!" looks complex, but it is one of the first guesses in any dictionary.

3.3.3 Storing passwords on the server



- A **salt** is a random value, different for each user, stored next to the hash. Two users with the same password get different hashes, and pre-computed (rainbow) tables become useless.
- A **slow hash** repeats work many times (PBKDF2, bcrypt) or also uses much memory (scrypt, Argon2). The user waits 0.1 second once; the attacker must wait 0.1 second for *each* guess.

Lab — Salted, slow password hashing `(c03_password.py)`

```python
# Chapter 3: storing passwords safely with salt + slow hash
import hashlib, os, hmac

def hash_password(pw, salt=None, rounds=200_000):
    salt = salt or os.urandom(16)
    h = hashlib.pbkdf2_hmac("sha256", pw.encode(), salt, rounds)
    return salt, h

def check_password(pw, salt, stored):
    _, h = hash_password(pw, salt)
    return hmac.compare_digest(h, stored)

salt, stored = hash_password("Cairo#2026")
print("salt  :", salt.hex())
print("stored:", stored.hex()[:32], "...")
print("right password:", check_password("Cairo#2026", salt, stored))
print("wrong password:", check_password("cairo2026", salt, stored))

# Same password, new salt -> different hash
```

```
_, other = hash_password("Cairo#2026")
print("same hash for two users?", other == stored)
```

Output:

```
salt   : dd821ce8c6a2d74631f2550e340f3cb8
stored: dc81e9600718f911e7ecdff80c7e602e ...
right password: True
wrong password: False
same hash for two users? False
```

Be careful

Never invent your own password scheme and never store passwords with encryption that can be reversed. The server does not need to know the password, only to check it.

3.4 One-time passwords

A one-time password (OTP) is good for only one login. If an attacker sees it, it is already useless.

- **HOTP** (RFC 4226): code $=$ Truncate(HMAC$(K,$ counter$)$).
- **TOTP** (RFC 6238): the counter is the time: counter $= \lfloor t/30 \rfloor$. A new 6-digit code every 30 seconds. This is what Google Authenticator and Microsoft Authenticator do.

The phone and the server share a secret K once (usually by scanning a QR code). After that, both compute the same code without talking to each other.

Lab — TOTP like Google Authenticator `(c03_totp.py)`

```
# Chapter 3: one-time codes (TOTP, RFC 6238) like Google Authenticator
import hmac, hashlib, struct, time

def totp(secret, t=None, step=30, digits=6):
    counter = int((t if t is not None else time.time()) // step)
    mac = hmac.new(secret, struct.pack(">Q", counter), hashlib.sha1).digest()
    off = mac[-1] & 0x0F
    code = struct.unpack(">I", mac[off:off + 4])[0] & 0x7FFFFFFF
    return str(code % 10**digits).zfill(digits)

secret = b"12345678901234567890"           # RFC 6238 test key
print("t=59        :", totp(secret, 59, digits=8))   # expected 94287082
print("t=1111111109:", totp(secret, 1111111109, digits=8))  # 07081804
print("now         :", totp(secret))
```

Output:

```
t=59        : 94287082
t=1111111109: 07081804
now         : 264167
```

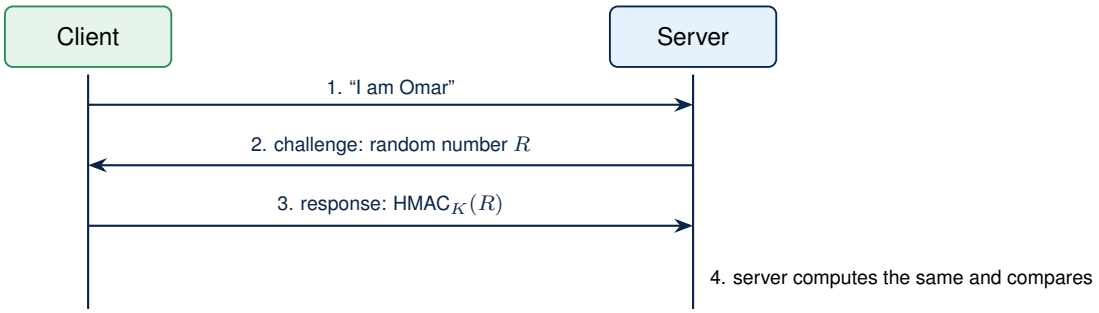
The first two outputs match the official test values in RFC 6238, so our code is correct.

Phishing-resistant MFA

TOTP codes can still be phished: a fake site asks for the code and uses it within 30 seconds. **FIDO2** / **WebAuthn** security keys and *passkeys* stop this. The key signs a challenge together with the real web site name, so a signature made for a fake site is useless on the real one. Passkeys are now the strongest common login method.

3.5 Challenge–response

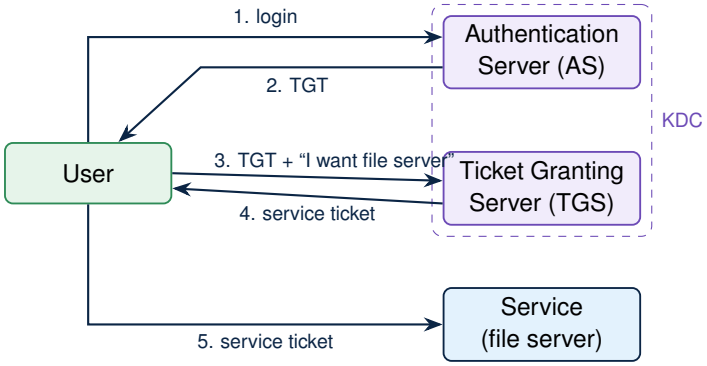
Sending a password over the network is dangerous even if it is hashed, because the attacker can *replay* the hash. A better way is challenge–response:



The secret K never travels, and an old response is useless because the next challenge R is different. The value R is called a **nonce** (number used once).

3.6 Kerberos and single sign-on

In a big network, a user should log in once and then use many services (file server, printer, email) without typing the password again. **Kerberos** (used by Microsoft Active Directory) does this with *tickets*.



1. The user logs in once to the Authentication Server and gets a **Ticket Granting Ticket (TGT)**.
2. For each service, the user shows the TGT to the TGS and gets a **service ticket**.
3. The service checks the ticket. The password is never sent over the network.

Tickets have timestamps and short lifetimes (often 10 hours), which stops replay. Kerberos needs the clocks of all machines to be close to each other.

On the web, single sign-on is done with **SAML**, **OAuth 2.0** and **OpenID Connect** (“Sign in with Google”).

3.7 Access control models

After we know *who* the user is, we must decide *what* the user can do. The rules can be shown as an access control matrix:

	grades.xlsx	exam.pdf	printer
Dr. Hany (teacher)	read, write	read, write	print
Omar (student)	read (own row)	–	print
IT admin	–	–	manage

A column of this matrix is an **access control list (ACL)**; a row is a **capability list**.

Model	Who decides?	Example
DAC (Discretionary)	The owner of the object.	Linux file permissions, sharing a Google Doc.
MAC (Mandatory)	The system, using security labels (Top Secret, Secret, . . .).	Military systems, SELinux.
RBAC (Role-based)	Rights are given to roles; users get roles.	"Teacher", "Student", "Admin" roles.
ABAC (Attribute-based)	Rules on attributes of user, object and context.	"Teachers may edit grades only from college network during work hours."

Bell–LaPadula and Biba

In military MAC, the **Bell–LaPadula** model protects confidentiality with two rules: *no read up* (a Secret user cannot read Top Secret) and *no write down* (a Top Secret user cannot write into a Secret file, so secrets do not leak down). The **Biba** model protects integrity with the opposite rules: *no read down, no write up*.

Chapter summary

- Authentication factors: know, have, are. MFA mixes different factors.
- Store passwords with a salt and a slow hash (Argon2, bcrypt, scrypt, PBKDF2).
- TOTP gives one-time codes from HMAC and time; FIDO2 / passkeys resist phishing.
- Kerberos gives single sign-on with tickets; access is controlled with DAC, MAC, RBAC or ABAC.

Exercises

1. **[B]** Is "password + security question" two-factor authentication? Explain.
2. **[B]** Compute the entropy of a 10-character password made of random digits only.
3. **[I]** Change the number of rounds in the password lab to 1, 100 000 and 1 000 000 and measure the time. What does this mean for the attacker?
4. **[I]** Design RBAC roles for a hospital: doctor, nurse, receptionist, patient.
5. **[E]** Under Bell–LaPadula, can a Secret user write to a Top Secret file? Can he read it? Explain why.
6. **[E]** Extend the TOTP lab to accept codes from the previous and next 30-second window. Why do real servers do this, and what is the security cost?

CHAPTER 4

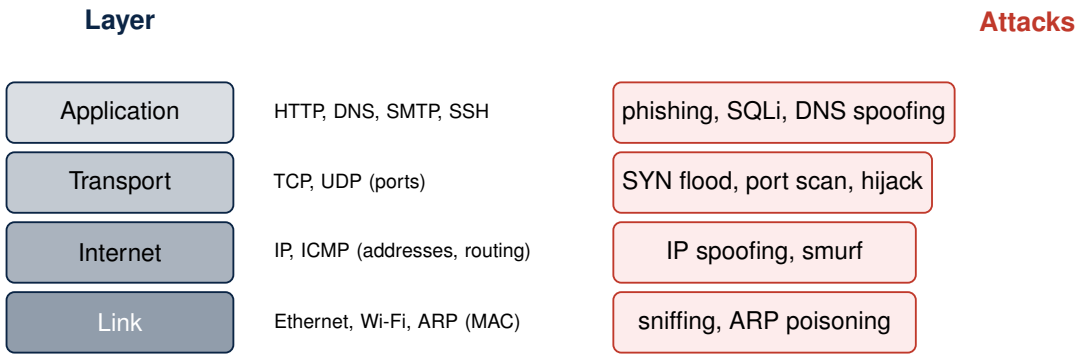
Network Fundamentals and Attacks

What you will learn

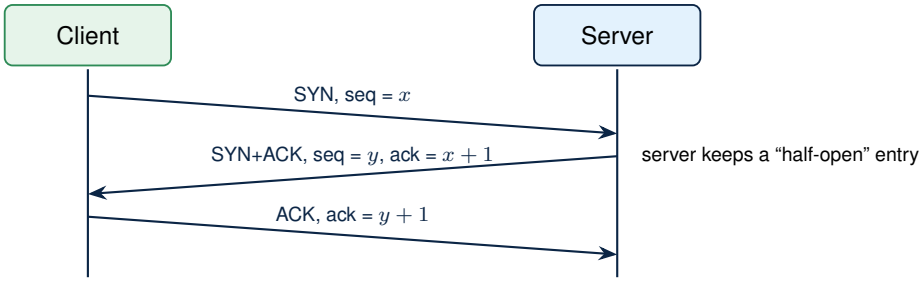
- Review the TCP/IP layers and see where each attack happens.
- Read IP and TCP headers byte by byte in Python.
- Understand sniffing, ARP poisoning, IP spoofing and DNS attacks.
- Understand scanning, session hijacking and DoS / DDoS attacks.
- Write a simple detector for SYN floods and port scans.

4.1 A short review of TCP/IP

The Internet protocols were designed in the 1970s for a small group of trusted researchers. Security was *not* a design goal. Most network attacks use this original trust.

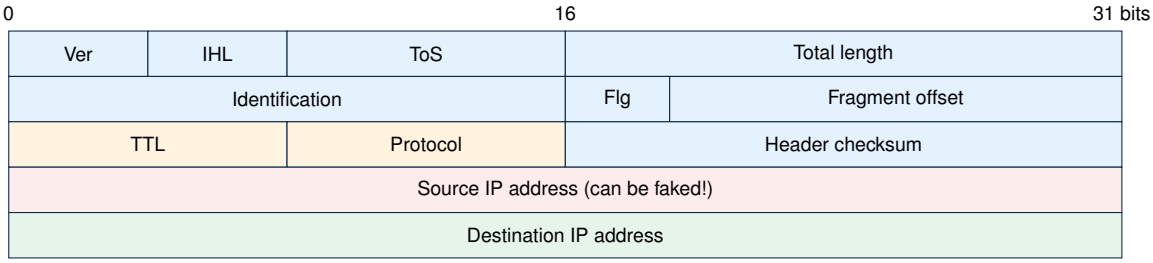


4.1.1 The TCP three-way handshake



The sequence numbers x and y must be random. If an attacker can guess them, he can inject fake data into the connection.

4.1.2 Reading a packet



The IP header has no authentication. Anyone who can send raw packets can write *any* source address. This is the root of IP spoofing.

Lab — Parse IPv4 and TCP headers `(c04_parse.py)`

```
# Chapter 4: read IPv4 and TCP headers from raw bytes
import struct, socket

raw = bytes.fromhex(
    "4500003c1c4640004006b1e6c0a80001c0a800c7"   # IPv4 header (20 bytes)
    "a2fe0050000003e800000000a0027210e2a90000"   # TCP header (first 20 bytes)
)
ver_ihl, tos, total, ident, frag, ttl, proto, csum, src, dst = \
    struct.unpack("!BBHHHBBH4s4s", raw[:20])
print("Version :", ver_ihl >> 4, " Header len:", (ver_ihl & 15) * 4, "bytes")
print("TTL     :", ttl, " Protocol:", proto, "(6 = TCP)")
print("From    :", socket.inet_ntoa(src), "->", socket.inet_ntoa(dst))

sport, dport, seq, ack, off_flags = struct.unpack("!HHIIH", raw[20:34])
flags = off_flags & 0x3F
names = [n for bit, n in [(1,"FIN"),(2,"SYN"),(4,"RST"),(8,"PSH"),(16,"ACK"),(32,"URG")] if
    flags & bit]
print("Ports   :", sport, "->", dport, " SEQ:", seq, " Flags:", names)
```

Output:

```
Version : 4  Header len: 20 bytes
TTL     : 64  Protocol: 6 (6 = TCP)
From    : 192.168.0.1 -> 192.168.0.199
Ports   : 41726 -> 80  SEQ: 1000  Flags: ['SYN']
```

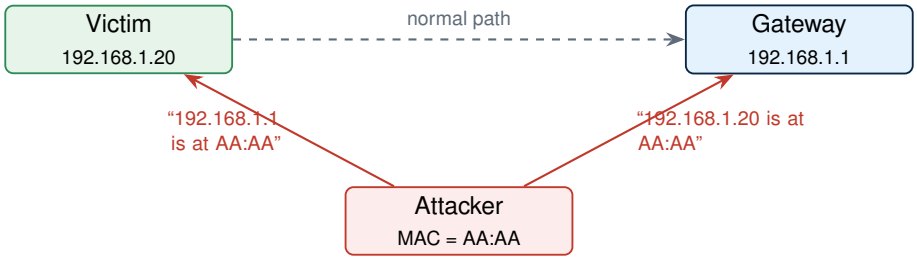
4.2 Sniffing

Sniffing means capturing packets that pass on the network. On an old hub or on open Wi-Fi, every machine sees all traffic. Tools like **Wireshark** and **tcpdump** are used by both admins (for debugging) and attackers (to steal passwords from old protocols like Telnet, FTP and plain HTTP).

- **Defense:** encrypt everything (TLS, SSH, VPN). Use switches, not hubs, and secure Wi-Fi.

4.3 ARP poisoning

Inside a LAN, machines use ARP to find the MAC address of an IP address: “Who has 192.168.1.1? Tell me.” Any machine can answer, and the asker believes the answer, even an answer that it did not ask for.



Now both the victim and the gateway send their traffic to the attacker, who forwards it. This is a man-in-the-middle (MITM) attack.

- **Defense:** Dynamic ARP Inspection on switches, static ARP entries for critical servers, port security, and encryption (so the MITM sees nothing useful).

4.4 IP spoofing

The attacker puts a fake source IP in his packets. He cannot normally receive the replies (they go to the fake address), so spoofing is mostly used in **DoS attacks** and **reflection attacks**.

- **Defense:** ingress / egress filtering (BCP 38): an ISP or a company router drops outgoing packets whose source address does not belong to its own network.

4.5 DNS attacks

DNS translates names (www.mtc.edu.eg) to IP addresses. If an attacker changes the answer, the victim goes to the attacker’s server while the address bar shows the right name.

- **DNS spoofing / cache poisoning:** the attacker sends fake replies to a DNS resolver before the real reply arrives. The fake answer is saved (cached) and given to many users. In 2008, Dan Kaminsky showed a fast way to do this, which forced a global patch (random source ports).
- **DNS hijacking:** changing DNS settings on a home router or at the domain registrar.
- **Defense:** DNSSEC (signed DNS answers), random ports and query IDs, encrypted DNS (DoH, DoT), and of course TLS certificates, which reveal a fake server.

4.6 Reconnaissance and scanning

Before attacking, attackers collect information:

- **Passive:** public web sites, social media, DNS records, WHOIS, job adverts (“we need an expert in Cisco ASA”).
- **Active:** ping sweeps to find live hosts, then **port scanning** to find open services, then **banner grabbing** to find software versions.

Scan type	How it works
TCP connect	Completes the full handshake. Simple but logged by the target.
SYN (half-open)	Sends SYN; SYN+ACK means open, RST means closed. Never completes.
UDP scan	No reply often means open; ICMP “port unreachable” means closed.

The tool **Nmap** is the standard scanner. Admins should scan *their own* networks often, to find services that should not be open.

4.7 Session hijacking

After a user logs in, the server trusts a *session*: a TCP connection or a web cookie. If the attacker steals the session cookie (by sniffing plain HTTP, or by XSS, Chapter 9), he becomes the user without knowing the password.

- **Defense:** HTTPS everywhere, cookies with the `Secure` and `HttpOnly` flags, short session time, new session ID after login.

4.8 Denial of service

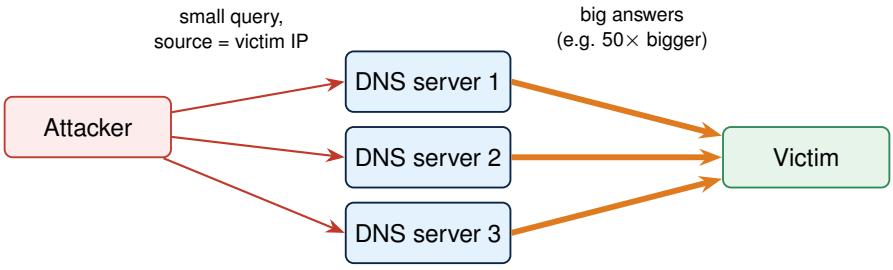
A DoS attack attacks *availability*. A **DDoS** (distributed DoS) uses thousands of machines (a **botnet**).

4.8.1 SYN flood

The attacker sends many SYN packets with fake source addresses. The server answers with SYN+ACK and keeps a half-open entry for each one, waiting for an ACK that never comes. Its table fills up and real users cannot connect.

Defense: SYN cookies. The server keeps no state. It encodes the connection data inside its own sequence number y (a keyed hash of the addresses, ports and time). Only when the ACK returns with $y + 1$ does it build the connection.

4.8.2 Reflection and amplification



The attacker sends small requests with the *victim's* address as source to open servers (DNS, NTP, Memcached). The servers send large replies to the victim. The **amplification factor** is reply size divided by request size: about 50 for DNS, up to about 500 for old NTP, and thousands for open Memcached servers.

Amplification arithmetic

An attacker has 1 000 bots, each with 2 Mbit/s upload. Total: 2 Gbit/s. With DNS amplification ($\times 50$), the victim receives up to $2 \times 50 = 100$ Gbit/s, enough to fill the Internet link of a large company.

4.8.3 Defending against DDoS

- Over-provision bandwidth and use **scrubbing services** or CDNs that absorb traffic.
- Rate-limiting, SYN cookies, and dropping spoofed traffic (BCP 38).
- Close open resolvers and unused UDP services so your servers are not used as reflectors.
- Have a DDoS response plan with your ISP *before* the attack.

Lab — Detect a SYN flood and a port scan `(c04_syn_detect.py)`

```
# Chapter 4: detect a SYN flood and a port scan from connection records
from collections import Counter, defaultdict

```

```python
# (source IP, destination port, flag) -- a small fake capture
events = [("10.0.0.5", 80, "SYN")] * 120 + [("10.0.0.5", 80, "ACK")] * 3
events += [("10.0.0.9", p, "SYN") for p in range(20, 60)]
events += [("10.0.0.7", 443, "SYN"), ("10.0.0.7", 443, "ACK")] * 10

syn = Counter(ip for ip, _, f in events if f == "SYN")
ack = Counter(ip for ip, _, f in events if f == "ACK")
ports = defaultdict(set)
for ip, port, f in events:
    if f == "SYN":
        ports[ip].add(port)

for ip in syn:
    ratio = syn[ip] / max(ack[ip], 1)
    alert = []
    if syn[ip] > 100 and ratio > 10:
        alert.append("SYN FLOOD")
    if len(ports[ip]) > 20:
        alert.append("PORT SCAN")
    print(f"{ip:10} SYN={syn[ip]:4} ACK={ack[ip]:3} ports={len(ports[ip]):3}",
          ", ".join(alert) or "normal")
```

Output:

```
10.0.0.5   SYN= 120 ACK=  3 ports=  1 SYN FLOOD
10.0.0.9   SYN=  40 ACK=  0 ports= 40 PORT SCAN
10.0.0.7   SYN=  10 ACK= 10 ports=  1 normal
```

Chapter summary

- TCP/IP was not built for security: addresses can be faked and ARP/DNS answers are trusted.
- Sniffing and MITM (ARP poisoning) steal data; encryption is the main defense.
- Scanning is the first step of most attacks; scan your own network first.
- DoS attacks availability: SYN floods, reflection and amplification. Use SYN cookies, filtering and scrubbing.

Exercises

1. **[B]** At which TCP/IP layer does each attack work: ARP poisoning, SYN flood, DNS spoofing?
2. **[B]** Why can an attacker who spoofs his IP address not complete a TCP handshake?
3. **[I]** Change the bytes in the packet-parsing lab so that the TCP flags are SYN+ACK and the destination port is 443.
4. **[I]** A DNS query is 60 bytes and the answer is 3 000 bytes. What is the amplification factor? How many 1 Mbit/s bots make a 10 Gbit/s attack?
5. **[E]** Explain how SYN cookies let a server answer without saving state. What TCP options are lost?
6. **[E]** Add a third rule to the detector: alert if one source sends to more than 10 different destination IPs in the record (a “horizontal scan”).

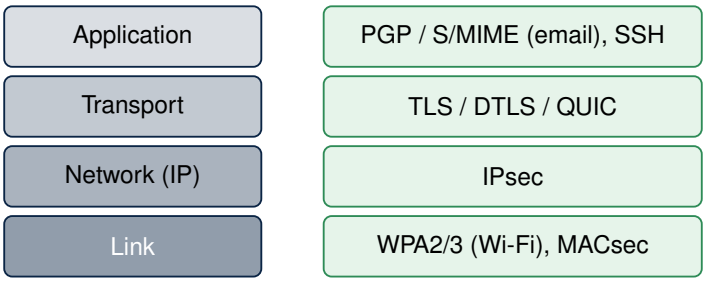
CHAPTER 5

Network Security Protocols

What you will learn

- See where security can be added in the protocol stack.
- Understand TLS 1.3: the handshake, keys and certificates.
- Understand SSH and its key-based login.
- Understand IPsec: AH, ESP, transport and tunnel modes, IKE.
- Compare the three protocols and choose the right one.

5.1 Where to put security?



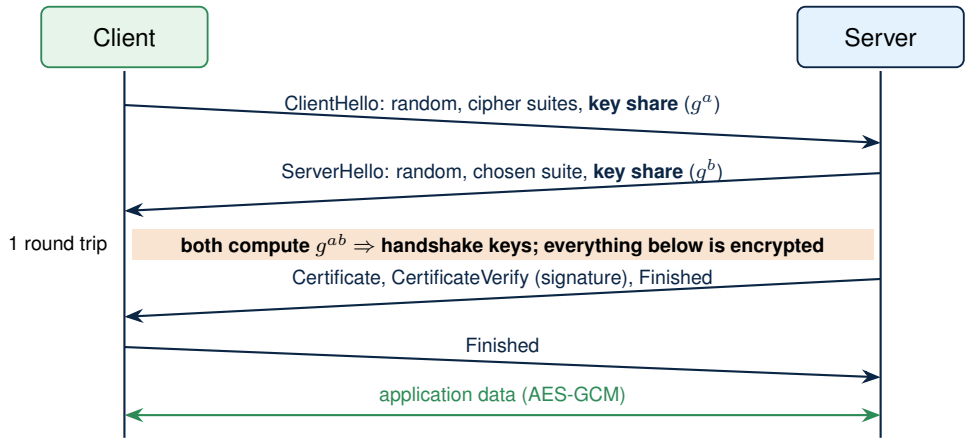
The lower the layer, the more traffic is protected without changing applications. The higher the layer, the more the protection understands the data (for example, it knows the user name).

5.2 TLS: Transport Layer Security

TLS is the “S” in HTTPS. It protects web, email, banking apps and most API traffic. The old name is SSL; all SSL versions and TLS 1.0 / 1.1 are broken and must be disabled. Today use **TLS 1.2** and preferably **TLS 1.3** (2018).

TLS gives three things: **confidentiality** (AES-GCM or ChaCha20), **integrity** (AEAD tags) and **server authentication** (certificates). Client authentication is optional.

5.2.1 The TLS 1.3 handshake



Step by step:

1. The client sends its supported algorithms and an ephemeral (temporary) Diffie–Hellman public value (usually X25519).
2. The server picks the algorithms and sends its own DH value. Both now compute the same secret, and derive keys with HKDF.
3. The server proves its identity: it sends its **certificate** and a **signature** over the whole handshake with its private key. This stops the MITM attack on DH that we saw in Chapter 2.
4. Both send `Finished`: a MAC over all handshake messages, so any change by an attacker is detected.

Forward secrecy

Because TLS 1.3 uses a *new* DH key for every connection and then deletes it, stealing the server's private key next year does **not** let an attacker decrypt traffic he recorded today. This property is called **forward secrecy**. TLS 1.3 removed the old RSA key exchange, which did not have it.

5.2.2 Cipher suites

A TLS 1.3 cipher suite such as `TLS_AES_128_GCM_SHA256` names the AEAD cipher (AES-128-GCM) and the hash for key derivation (SHA-256). In TLS 1.2 the name also includes the key exchange and the signature, for example `ECDHE-RSA-AES256-GCM-SHA384`.

Lab — A safe TLS client context `(c05_tls.py)`

```python
# Chapter 5: look at a safe TLS client setup in Python
import ssl

ctx = ssl.create_default_context()          # checks certificate + host name
ctx.minimum_version = ssl.TLSVersion.TLSv1_2
print("OpenSSL         :", ssl.OPENSSL_VERSION)
print("Verify mode     :", ctx.verify_mode.name)
print("Check host name :", ctx.check_hostname)
print("Lowest version  :", ctx.minimum_version.name)
names = [c["name"] for c in ctx.get_ciphers()]
print("Some cipher suites:")
for n in names[:5]:
    print("   ", n)
# To connect:  ctx.wrap_socket(sock, server_hostname="www.example.com")
```

Output:

```
OpenSSL         : OpenSSL 3.5.9 29 Sep 2026
Verify mode     : CERT_REQUIRED
```

```
Check host name : True
Lowest version  : TLSv1_2
Some cipher suites:
    TLS_AES_256_GCM_SHA384
    TLS_CHACHA20_POLY1305_SHA256
    TLS_AES_128_GCM_SHA256
    ECDHE-ECDSA-AES256-GCM-SHA384
    ECDHE-RSA-AES256-GCM-SHA384
```

Common TLS mistakes

- Turning off certificate checking (`verify=False` in Python `requests`). This removes *all* protection against MITM.
- Allowing old versions (SSL 3.0, TLS 1.0) or weak ciphers (RC4, 3DES, export ciphers).
- Expired certificates: users learn to click "continue anyway".
- Serving the login page over HTTPS but other pages over HTTP: cookies can still be stolen. Use HSTS to force HTTPS.

Famous TLS attacks

Heartbleed (2014) was a bug in OpenSSL code (not in the protocol): a missing length check let anyone read 64 KB of server memory, including private keys. **POODLE** (2014) broke SSL 3.0 padding. **BEAST**, **Lucky13** and **ROBOT** attacked CBC mode and RSA key exchange. TLS 1.3 removed all of these old features.

5.3 SSH: Secure Shell

SSH replaces Telnet, rlogin and FTP for remote administration. Like TLS, it uses DH key exchange, then symmetric encryption. Its trust model is different: there is usually no CA.

- **Server authentication: TOFU** (trust on first use). The first time you connect, SSH shows the server key fingerprint and saves it in the file `known_hosts`. If the key changes later, SSH shows a big warning: a MITM may be present.
- **User authentication:** password, or better, a **key pair**. The public key is put on the server in the file `~/.ssh/authorized_keys`; the private key stays on your laptop.

```
ssh-keygen -t ed25519 -C "omar@mtc"      # make a key pair
ssh-copy-id omar@server.mtc.local        # install the public key
ssh omar@server.mtc.local                # login with the key, no password
```

Hardening an SSH server (`/etc/ssh/sshd_config`): disable root login (`PermitRootLogin no`), disable passwords (`PasswordAuthentication no`), allow only some users, and use a tool like fail2ban to block brute force.

5.4 IPsec

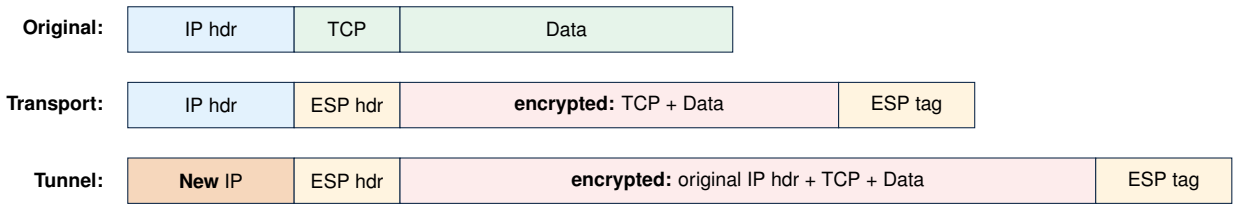
IPsec protects **all** IP traffic between two hosts or two networks. It is the classic technology for site-to-site VPNs (Chapter 7).

5.4.1 AH and ESP

- **AH** (Authentication Header): integrity and authentication only, no encryption. Rarely used today, and it does not work through NAT.

- **ESP** (Encapsulating Security Payload): encryption *and* integrity. This is what is used in practice.

5.4.2 Transport mode and tunnel mode



- **Transport mode:** protects only the payload; the original IP header stays. Used host-to-host.
- **Tunnel mode:** the whole original packet is encrypted and put inside a new packet. Used gateway-to-gateway (VPN). An observer sees only the two gateway addresses, not the real hosts inside.

5.4.3 Security associations and IKE

A **Security Association (SA)** is a one-way agreement: which algorithms, which keys, which lifetime. Each SA has a number called the SPI, written in every ESP header. Two SAs are needed for two-way traffic.

SAs are created automatically by **IKEv2** (Internet Key Exchange): the two peers do a DH exchange, authenticate each other (with certificates or a pre-shared key) and derive the keys for ESP.

Replay protection

Every ESP packet carries a sequence number. The receiver keeps a sliding window (often 64 packets) and drops any number it has already seen. You will build this in the Chapter 7 lab.

5.5 Comparing TLS, SSH and IPsec

	TLS	SSH	IPsec
Layer	Transport (above TCP/UDP)	Application	Network (IP)
Protects	One application connection	Remote shell, file copy, tunnels	All IP traffic
Server identity	CA certificates	TOFU fingerprint	Certificates or PSK
Needs app change?	Yes (app uses TLS)	–	No
Typical use	HTTPS, email, APIs	Admin access	Site-to-site VPN

Chapter summary

- TLS 1.3: ephemeral DH + certificate signature + AEAD; one round trip; forward secrecy.
- Never disable certificate checking; disable SSL and TLS 1.0/1.1.
- SSH: TOFU host keys and key-based user login; harden the server.
- IPsec ESP in tunnel mode protects whole networks; IKEv2 sets up the keys.

Exercises

1. **[B]** What three security services does TLS give? Which one is missing if the client does not check the certificate?
2. **[B]** What does SSH do when the server key changes? Why?

3. **[I]** Explain forward secrecy with an example of an attacker who records traffic in 2026 and steals the key in 2027.
4. **[I]** Draw the packet for ESP tunnel mode between two gateways 41.1.1.1 and 156.2.2.2 carrying a packet from 10.1.0.5 to 10.2.0.8.
5. **[E]** Use Python's `ssl` module to connect to a site you choose and print the TLS version, cipher suite and certificate issuer.
6. **[E]** Why does AH not work through NAT, while ESP (with UDP encapsulation) does?

CHAPTER 6

Firewalls and Intrusion Detection

What you will learn

- Understand firewall types: packet filter, stateful, proxy and next-generation.
- Write firewall rules and understand rule order.
- Design a network with a DMZ and segments.
- Understand IDS and IPS: signature and anomaly detection, false alarms.
- Build a small firewall and a small IDS in Python.

6.1 What is a firewall?

A firewall is a device or program that sits between networks of different trust and allows or blocks traffic based on rules. Think of it as the gate guard of the college: he checks every person who enters or leaves against a list.

Default deny

A good firewall policy says: **block everything, then allow only what is needed**. The opposite (allow all, block known bad) always misses something.

6.2 Types of firewalls

6.2.1 Packet filters (stateless)

Each packet is checked alone against rules on: source / destination IP, protocol, source / destination port, and interface. They are very fast (often inside routers as ACLs), but they do not remember connections. For example, to allow web replies they must allow *all* packets from port 80, which an attacker can use.

6.2.2 Stateful firewalls

A stateful firewall keeps a **connection table**. When an inside user opens a connection, the firewall remembers it, and allows only the replies that belong to it.

Source	Dest	Ports	Proto	State
10.0.0.15	142.250.1.1	51544 → 443	TCP	ESTABLISHED
10.0.0.22	8.8.8.8	60001 → 53	UDP	NEW

A packet from outside that does not match any row is dropped. Almost all modern firewalls are stateful.

6.2.3 Application proxies

A proxy (application-level gateway) stops the connection, reads the full application message (for example the HTTP request), checks it, and opens a *new* connection to the server. It understands the content, so it can block a bad URL or a file type, but it is slower. A **Web Application Firewall (WAF)** is a proxy specialized in blocking web attacks like SQL injection.

6.2.4 Next-generation firewalls (NGFW)

An NGFW joins all the above: stateful filtering, application recognition (“this is WhatsApp”, not just “port 443”), user identity, IPS, TLS inspection and threat intelligence feeds.

Type	Layer	Speed	Knows about
Packet filter	3–4	very high	addresses, ports
Stateful	3–4	high	+ connections
Proxy / WAF	7	lower	+ application content
NGFW	3–7	medium	+ apps, users, threats

6.3 Writing firewall rules

Rules are checked from top to bottom; the **first matching rule wins**. So specific rules must come before general rules.

iptables rules for a Linux web server

```
iptables -P INPUT DROP                                      # default deny
iptables -A INPUT -i lo -j ACCEPT                           # local traffic
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -p tcp -s 192.168.1.0/24 --dport 22 -j ACCEPT  # SSH from LAN
iptables -A INPUT -p tcp --dport 443 -j ACCEPT              # HTTPS for all
iptables -A INPUT -p tcp --dport 22 -m recent --set         # (rate limit idea)
```

Rule order

If you put “ALLOW any any” at the top, nothing below it will ever run. Also, old rules are often forgotten (“temporary” rules from three years ago). Review the rule set regularly and write a comment for every rule.

Lab — A first-match packet filter (c06_firewall.py)

```python
# Chapter 6: a first-match packet filter
import ipaddress

RULES = [  # action, source, dest port, protocol
    ("ALLOW", "192.168.1.0/24", 22,  "tcp"),   # SSH only from inside
    ("DENY",  "0.0.0.0/0",      22,  "tcp"),
    ("ALLOW", "0.0.0.0/0",      443, "tcp"),   # HTTPS for everyone
    ("ALLOW", "0.0.0.0/0",      53,  "udp"),   # DNS
    ("DENY",  "0.0.0.0/0",      None, None),   # default deny
]

def decide(src, port, proto):
    for i, (act, net, p, pr) in enumerate(RULES, 1):
        if ipaddress.ip_address(src) in ipaddress.ip_network(net) \
           and p in (None, port) and pr in (None, proto):
            return act, i

for pkt in [("192.168.1.20", 22, "tcp"), ("41.33.10.5", 22, "tcp"),
            ("41.33.10.5", 443, "tcp"), ("41.33.10.5", 3389, "tcp")]:
    act, rule = decide(*pkt)
    print(f"{pkt[0]:14} port {pkt[1]:5}/{pkt[2]} -> {act:5} (rule {rule})")
```

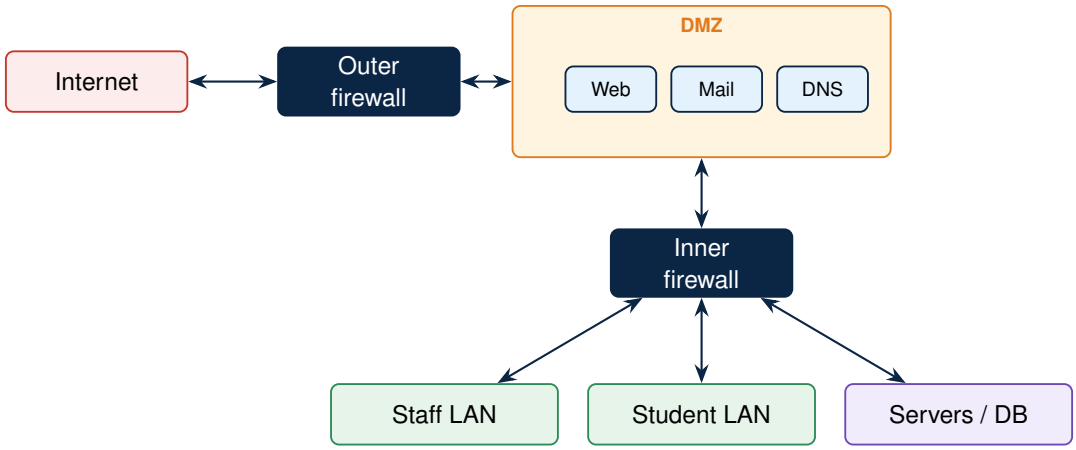
Output:

```
192.168.1.20   port    22/tcp -> ALLOW (rule 1)
41.33.10.5     port    22/tcp -> DENY  (rule 2)
41.33.10.5     port   443/tcp -> ALLOW (rule 3)
```

```
41.33.10.5      port  3389/tcp -> DENY  (rule 5)
```

6.4 Network design: DMZ and segmentation

Servers that must be reached from the Internet (web, mail, DNS) should not be in the same network as the internal users. We put them in a **DMZ** (demilitarized zone).



Rules for this design:

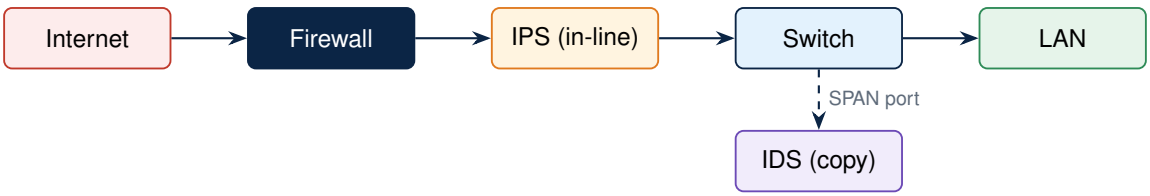
- Internet → DMZ: only the needed ports (443 to web, 25 to mail, 53 to DNS).
- Internet → inside: nothing.
- DMZ → inside: almost nothing (maybe the web server to one database port).
- Students → servers: only the services they need.

Segmentation (VLANs and internal firewalls) limits *lateral movement*: if one student PC is infected, the malware cannot reach the grades database directly.

6.5 Intrusion detection and prevention

A firewall is a *guard at the gate*; an IDS is a *camera with an alarm*. It watches traffic or logs and raises an alert when something looks like an attack.

- **NIDS** (network IDS) watches packets on the network, for example on a mirror (SPAN) port of a switch.
- **HIDS** (host IDS) watches one computer: log files, changes in system files, running processes.
- **IPS** (intrusion prevention system) sits *in-line* and can drop the bad packets, not only alert.



6.5.1 Detection methods

- **Signature-based (misuse):** looks for known patterns, like antivirus. Few false alarms, but it cannot see new (zero-day) attacks.

- **Anomaly-based:** learns what “normal” looks like (traffic volume, login times, protocols), then alerts on things far from normal. It can find new attacks, but gives more false alarms. Machine learning is often used here.
- **Specification / stateful protocol analysis:** checks that traffic follows the protocol rules.

A Snort / Suricata rule looks like this:

```
alert tcp any any -> $HOME_NET 80 (msg:"SQL injection attempt";
      content:"' OR 1=1"; nocase; sid:1000001; rev:1;)
```

Lab — A tiny signature IDS (c06_ids.py)

```
# Chapter 6: a tiny signature-based IDS for web logs
import re

SIGNATURES = {
    "SQL injection":  re.compile(r"('|%27)\s*(or|and)\s+\d+=\d+|union\s+select", re.I),
    "XSS":            re.compile(r"<script|onerror\s*=", re.I),
    "Path traversal": re.compile(r"\.\./|%2e%2e%2f", re.I),
}
logs = [
    "GET /index.html",
    "GET /login?user=admin' OR 1=1--",
    "GET /search?q=<script>alert(1)</script>",
    "GET /files?name=../../etc/passwd",
    "GET /products?id=7",
]
for line in logs:
    hits = [name for name, rx in SIGNATURES.items() if rx.search(line)]
    print(("ALERT " + ", ".join(hits)) if hits else "ok   ", "|", line)
```

Output:

```
ok    | GET /index.html
ALERT SQL injection | GET /login?user=admin' OR 1=1--
ALERT XSS | GET /search?q=<script>alert(1)</script>
ALERT Path traversal | GET /files?name=../../etc/passwd
ok    | GET /products?id=7
```

6.5.2 Measuring an IDS

	Alert	No alert
Real attack	True positive (TP)	False negative (FN)
Normal traffic	False positive (FP)	True negative (TN)

$$\text{Detection rate} = \frac{TP}{TP + FN}, \qquad \text{False alarm rate} = \frac{FP}{FP + TN}.$$

The base-rate fallacy

Suppose an IDS detects 99% of attacks and has only a 1% false alarm rate. In a day there are 1 000 000 normal events and 10 real attacks. Alerts: $0.99 \times 10 \approx 10$ true, and $0.01 \times 1\,000\,000 = 10\,000$ false. So only about **1 alert in 1000 is real**! Analysts get tired and miss the real one (“alert fatigue”). Because attacks are rare, the false alarm rate must be extremely low.

6.6 Honeypots

A honeypot is a fake system that has no real users. So *any* connection to it is suspicious. Honeypots catch scanners and insiders, show attacker tools, and waste attacker time. A **honeytoken** is the same idea for data: a fake password or a fake database record that raises an alarm if anyone uses it.

Chapter summary

- Default deny; first matching rule wins; review rules often.
- Stateful firewalls track connections; proxies and WAFs understand content; NGFWs join everything.
- Use a DMZ for public servers and segment the inside network.
- IDS detects (signature or anomaly), IPS blocks; false alarms are the main practical problem.

Exercises

1. **[B]** What is the difference between a stateless and a stateful firewall? Give an example packet that only a stateful firewall can block.
2. **[B]** Why must the web server be in the DMZ and not in the staff LAN?
3. **[I]** Add rules to the firewall lab so that the DNS server 192.168.1.53 can receive UDP 53 from anywhere, but no other host can.
4. **[I]** An IDS has TP = 45, FN = 5, FP = 200, TN = 99 750. Compute the detection rate and the false alarm rate.
5. **[E]** Add a "command injection" signature to the IDS lab (for example `;cat` or `|ls`). Then write a log line that avoids your signature. What does this teach about signatures?
6. **[E]** Repeat the base-rate example with a false alarm rate of 0.01%. What fraction of alerts is now real?

CHAPTER 7

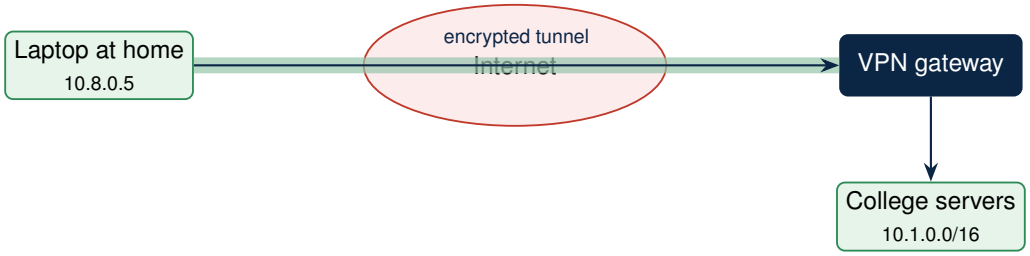
Virtual Private Networks (VPNs)

What you will learn

- Understand what a VPN is and what it does *not* protect.
- Know the two main VPN types: remote access and site-to-site.
- Compare IPsec/IKEv2, TLS-based VPNs (OpenVPN) and WireGuard.
- Compute the overhead of tunneling and understand MTU problems.
- Build a toy ESP tunnel with replay protection in Python.

7.1 What is a VPN?

A **Virtual Private Network** builds a protected "tunnel" over a public network (the Internet). Packets of the private network are encrypted and put inside normal Internet packets. To the users it looks as if they are on the same private LAN, even if they are in different cities.



A VPN gives:

- **Confidentiality** of traffic over the untrusted network (Wi-Fi in a café, the ISP).
- **Integrity and authentication** of the two ends of the tunnel.
- **Access** to internal resources as if you were inside.

What a VPN does not do

A VPN protects data only *inside the tunnel*. It does not remove malware from your laptop, it does not make a phishing site safe, and the VPN provider can see all your traffic after the tunnel ends. A commercial "privacy VPN" only moves trust from your ISP to the VPN company. Also, if an infected laptop connects by VPN, the malware is now *inside* the network.

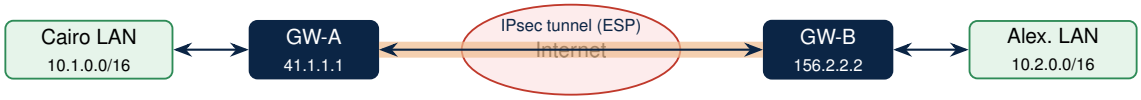
7.2 Types of VPN

7.2.1 Remote-access VPN

One user (laptop or phone) connects to the organization's gateway. Used by staff who work from home or travel. The user runs a VPN client and logs in, ideally with MFA.

7.2.2 Site-to-site VPN

Two gateways (routers or firewalls) build a permanent tunnel between two offices. Users do nothing special; their traffic is protected automatically.



7.2.3 Full tunnel and split tunnel

- **Full tunnel:** *all* traffic of the laptop goes through the VPN, even YouTube. The company can filter and inspect everything, but its Internet link is loaded.
- **Split tunnel:** only traffic to the company networks goes through the VPN; the rest goes directly to the Internet. Faster, but the laptop is connected to the Internet and to the company at the same time, so it can act as a bridge for an attacker.

7.3 VPN technologies

7.3.1 IPsec with IKEv2

We met IPsec in Chapter 5. For a VPN we use **ESP in tunnel mode**, with keys made by **IKEv2**. The IKE exchange has two phases:

1. **IKE SA:** the two peers do Diffie–Hellman and authenticate each other (certificates, pre-shared key, or EAP for users). This builds a protected control channel.
2. **Child SAs:** inside the control channel they agree on the ESP keys and on which traffic to protect (the *traffic selectors*, for example 10.1.0.0/16 ↔ 10.2.0.0/16).

IPsec is a standard, so a Cisco gateway can talk to a Fortinet or Linux (strongSwan) gateway. It is the normal choice for site-to-site VPNs. Its weak points are complex configuration and problems with NAT (solved by **NAT-T**: ESP inside UDP port 4500).

7.3.2 TLS-based VPNs (SSL VPN, OpenVPN)

These VPNs use TLS (Chapter 5) to build the tunnel, usually on TCP or UDP port 443. Because port 443 is open almost everywhere (it is normal HTTPS), they pass through hotel and airport firewalls easily. Many company “SSL VPN” portals also allow access to web applications from a browser with no client.

Be careful

SSL VPN gateways are on the Internet and are a favorite target. Several serious vulnerabilities in commercial VPN gateways have been used by attackers in recent years to enter company networks. Patch VPN gateways **immediately** when a security update appears.

7.3.3 WireGuard

WireGuard (in the Linux kernel since 2020) is a modern VPN with a very small code base (a few thousand lines, compared with hundreds of thousands for IPsec or OpenVPN implementations). It has *no* cipher negotiation: it always uses Curve25519, ChaCha20-Poly1305, BLAKE2s and the Noise protocol framework. Each peer is identified only by its public key.

A WireGuard configuration

```
[Interface]                          # on the laptop
PrivateKey = <laptop private key>
Address    = 10.8.0.5/24

[Peer]                               # the college gateway
PublicKey  = <gateway public key>
Endpoint   = vpn.example.edu:51820
AllowedIPs = 10.1.0.0/16             # split tunnel: only college traffic
```

Change `AllowedIPs` to `0.0.0.0/0` to make a full tunnel.

7.3.4 Comparison

	IPsec / IKEv2	OpenVPN (TLS)	WireGuard
Layer	Network (IP)	Uses TLS, user space	Network, in kernel
Ciphers	Negotiated (many)	Negotiated (TLS)	Fixed, modern
Code size	Very large	Large	Very small
Through firewalls	Needs UDP 500/4500	Easy (443)	Needs one UDP port
Speed	High (hardware support)	Medium	Very high
Best for	Site-to-site, standards	Remote users, restrictive networks	Modern remote access and servers

7.4 Tunnel overhead and MTU

A tunnel adds headers to every packet. Ethernet carries at most 1500 bytes of IP packet (the **MTU**). If the inner packet is 1500 bytes and we add 60 bytes of tunnel headers, the result does not fit and must be fragmented or dropped.

ESP tunnel overhead with AES-GCM

New outer IPv4 header	20 bytes
UDP header (NAT-T)	8 bytes
ESP header (SPI + sequence)	8 bytes
IV for AES-GCM	8 bytes
Padding + pad length + next header	2–5 bytes
Authentication tag (ICV)	16 bytes
Total	about 62–65 bytes

So the largest inner packet is about $1500 - 65 = 1435$ bytes. For TCP the safe MSS is $1435 - 40 = 1395$ bytes. WireGuard has about 60 bytes of overhead over IPv4 (80 over IPv6), so its default MTU is 1420.

Key idea

Many “the VPN connects but some web sites do not load” problems are MTU problems: small packets pass, big packets are dropped. The fix is to lower the tunnel MTU or to clamp the TCP MSS on the gateway.

7.5 Lab: a toy ESP tunnel

Our program imitates the main ideas of ESP:

- The header has an SPI and a sequence number. It is not encrypted, but it is *authenticated* (passed as associated data to AES-GCM), so an attacker cannot change it.
- The inner packet is encrypted with AES-GCM.
- The receiver keeps a replay window and drops any sequence number it has already seen.

Lab — Toy ESP with replay window (c07_esp.py)

```python
# Chapter 7: a toy ESP tunnel (IPsec idea) with a replay window
import os, struct
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

key = AESGCM.generate_key(bit_length=128)
SPI = 0x1001

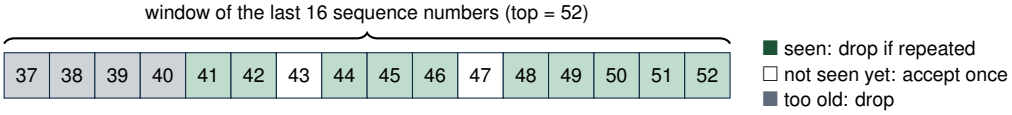
def esp_send(seq, inner_packet):
    hdr = struct.pack("!II", SPI, seq)          # SPI + sequence number
    nonce = os.urandom(12)
    return hdr + nonce + AESGCM(key).encrypt(nonce, inner_packet, hdr)

class Receiver:
    def __init__(self, window=64):
        self.top, self.seen, self.window = 0, set(), window
    def recv(self, pkt):
        hdr, nonce, body = pkt[:8], pkt[8:20], pkt[20:]
        spi, seq = struct.unpack("!II", hdr)
        if seq in self.seen or seq <= self.top - self.window:
            return "DROP (replay)"
        data = AESGCM(key).decrypt(nonce, body, hdr)  # also checks integrity
        self.seen.add(seq); self.top = max(self.top, seq)
        return "OK " + data.decode()

r = Receiver()
p1 = esp_send(1, b"10.1.0.5 -> 10.2.0.8: hello")
p2 = esp_send(2, b"10.1.0.5 -> 10.2.0.8: bye")
print(r.recv(p1)); print(r.recv(p2))
print(r.recv(p1))                    # attacker sends p1 again
```

Output:

```
OK 10.1.0.5 -> 10.2.0.8: hello
OK 10.1.0.5 -> 10.2.0.8: bye
DROP (replay)
```



VPNs and zero trust

Classic VPNs follow a “castle and moat” idea: once you are inside the tunnel, you are trusted on the whole network. Modern designs move to **zero trust network access (ZTNA)**: users connect to *one application at a time*, and every request is checked (identity, device health, context). We return to this in Chapter 11.

Chapter summary

- A VPN builds an encrypted tunnel over the Internet; it does not clean an infected device.
- Remote-access VPNs serve users; site-to-site VPNs join networks.
- IPsec/IKEv2 is the standard; TLS VPNs pass firewalls easily; WireGuard is small and fast.
- Tunnels add about 60 bytes per packet; watch the MTU.

Exercises

1. **[B]** Give one case where a remote-access VPN is the right choice and one where a site-to-site VPN is.
2. **[B]** Is your traffic private from the VPN provider? Explain.
3. **[I]** Compute the largest inner packet if the outer link has MTU 1492 (PPPoE) and the tunnel overhead is 73 bytes.
4. **[I]** What are the security risks of split tunneling? When would you accept them?
5. **[E]** In the ESP lab, an attacker changes the sequence number in the header of `p1` to 3 and sends it again. What happens and why? Test it.
6. **[E]** Change the lab so the receiver keeps only a 64-bit bitmap and the top number (like real IPsec), not a Python set.

CHAPTER 8

Wireless and Mobile Security

What you will learn

- Understand why wireless networks are harder to protect.
- Follow the history: WEP, WPA, WPA2 and WPA3.
- Understand the WPA2 4-way handshake and how keys are made.
- Know the main wireless attacks: evil twin, deauthentication, KRACK, offline cracking.
- Learn the basics of mobile, Bluetooth and cellular security.

8.1 Why wireless is different

In a wired network, an attacker must plug a cable into your switch. In a wireless network, the signal goes through walls into the street and the parking area. Anyone with an antenna can:

- **listen** to all frames (passive sniffing is invisible),
- **send** frames (fake access points, fake management frames),
- **jam** the radio channel (a physical DoS that no cryptography can stop).

So in wireless, *strong encryption and authentication are not optional*.

8.2 The history of Wi-Fi security



8.2.1 WEP: a lesson in bad design

WEP (Wired Equivalent Privacy) used the RC4 stream cipher with a key made from a 24-bit IV and the shared password: RC4$(IV \,\|\, K)$. It had many mistakes:

- The IV is only 24 bits, so after about $2^{12} \approx 5000$ packets an IV repeats with 50% chance (birthday paradox, Chapter 2). Same IV $\Rightarrow$ same key stream $\Rightarrow C_1 \oplus C_2 = P_1 \oplus P_2$.
- Integrity used CRC-32, which is linear: an attacker can flip bits and fix the CRC without the key.
- Weak RC4 keys (the FMS attack) let tools recover the key from a few tens of thousands of packets, today in under a minute.

Be careful

WEP must never be used. If an old device supports only WEP, put it in a separate, isolated network or replace it.

8.2.2 WPA and WPA2

WPA (2003) was a quick fix that could run on old WEP hardware (TKIP, still RC4). **WPA2** (2004) brought **AES-CCMP**: AES in counter mode with a CBC-MAC for integrity. WPA2 is still very common.

WPA2 has two modes:

- **Personal (PSK):** one shared passphrase for everybody. Used at home.
- **Enterprise (802.1X):** each user logs in with his own account through a RADIUS server, often using EAP-TLS (certificates) or PEAP (user name and password). Used in companies and universities (for example *eduroam*).

8.3 How WPA2 makes its keys

8.3.1 From passphrase to PMK

In WPA2-Personal, the **Pairwise Master Key (PMK)** is computed from the passphrase and the network name (SSID):

$$\text{PMK} = \text{PBKDF2}(\text{HMAC-SHA1},\ \text{passphrase},\ \text{SSID},\ 4096,\ 256\ \text{bits}).$$

The SSID acts as a salt, and 4096 rounds make each guess slower.

Lab — WPA2 passphrase to PMK (c08_pmk.py)

```python
# Chapter 8: how WPA2-Personal turns a passphrase into a key
import hashlib, time

def pmk(passphrase, ssid):
    return hashlib.pbkdf2_hmac("sha1", passphrase.encode(), ssid.encode(), 4096, 32)

# Test vector from IEEE 802.11i
print(pmk("password", "IEEE").hex())
# expected f42c6fc52df0ebef9ebb4b90b38a5f902e83fe1b135a70e23aed762e9710a12e

# Why weak passphrases are dangerous: guesses per second on one CPU core
t = time.time(); n = 0
while time.time() - t < 1:
    pmk(f"guess{n}", "MTC-Lab"); n += 1
print(f"about {n} guesses per second on this core")
```

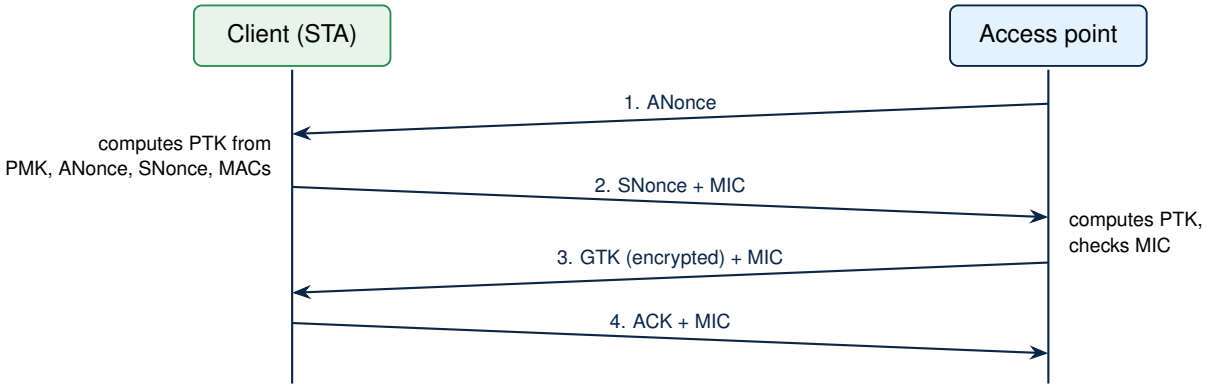
Output:

```
f42c6fc52df0ebef9ebb4b90b38a5f902e83fe1b135a70e23aed762e9710a12e
about 799 guesses per second on this core
```

The first line matches the official IEEE 802.11i test vector. The speed number is from a single slow CPU core; a modern GPU can test hundreds of thousands of passphrases per second.

8.3.2 The 4-way handshake

The PMK is never used directly. Each time a device joins, the device and the access point make a fresh **Pairwise Transient Key (PTK)** with the 4-way handshake:



$$\text{PTK} = \text{PRF}\big(\text{PMK},\ \text{ANonce},\ \text{SNonce},\ \text{MAC}_{AP},\ \text{MAC}_{STA}\big).$$

The PTK is split into a key for protecting the handshake messages (the MIC) and a key for encrypting data. The **GTK** (group key) protects broadcast traffic.

Why weak Wi-Fi passwords are dangerous

An attacker who captures one 4-way handshake has the nonces, the MAC addresses and a MIC. He can then guess passphrases **offline**: for each guess compute PMK $\rightarrow$ PTK $\rightarrow$ MIC and compare. No more contact with the network is needed and no one notices. A random passphrase of 12+ characters (or 4–5 random words) defeats this attack.

8.4 WPA3

WPA3 (2018) fixes the main problems of WPA2-Personal:

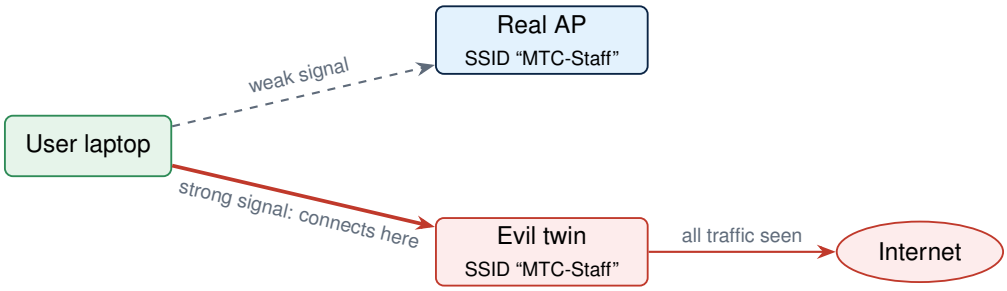
- **SAE** (Simultaneous Authentication of Equals, also called Dragonfly) replaces the PSK handshake. It is a *password-authenticated key exchange*: each guess requires a live exchange with the access point, so **offline dictionary attacks are not possible**.
- **Forward secrecy:** learning the password later does not decrypt old recorded traffic.
- **Protected Management Frames (PMF)** are mandatory, which stops deauthentication attacks.
- **WPA3-Enterprise 192-bit mode** uses GCMP-256 and stronger curves for government and military networks.
- **OWE (Enhanced Open):** encrypts open café networks without a password.

Dragonblood

In 2019, researchers found side-channel and downgrade attacks on early WPA3 implementations (“Dragonblood”): timing differences leaked information about the password, and “transition mode” (WPA2 and WPA3 together) allowed downgrade to WPA2. Implementations were fixed. The lesson: a good protocol can still be broken by a bad implementation.

8.5 Common wireless attacks

Attack	How it works	Defense
Evil twin / rogue AP	A fake access point with the same SSID and a stronger signal. Users connect to it; the attacker is a MITM.	WPA2/3-Enterprise with server certificate check; wireless IDS; VPN.
Deauthentication	Fake “deauth” frames kick users off; used to capture handshakes or push users to the evil twin.	PMF (802.11w), WPA3.
Offline cracking	Capture a WPA2-PSK handshake, guess offline.	Long random passphrase, WPA3-SAE.
KRACK (2017)	Replays message 3 of the 4-way handshake so the client reinstalls a key and reuses nonces.	Patch clients and APs.
WPS PIN attack	The 8-digit WPS PIN is checked in two halves: only about 11 000 tries.	Disable WPS.
Jamming	Radio noise on the channel.	Physical security, detection, wired backup.



8.6 Securing a Wi-Fi network: checklist

1. Use WPA3 (or WPA2-AES if needed). Never WEP or TKIP.
2. For organizations, use 802.1X / EAP-TLS and make clients *check the server certificate*.
3. For home, use a long random passphrase and disable WPS.
4. Change the default admin password of the router and update its firmware.
5. Put guests and IoT devices in a separate network (VLAN).
6. Use a wireless IDS to detect rogue access points.

8.7 Bluetooth and mobile security

8.7.1 Bluetooth

Bluetooth has its own pairing and encryption. Old “legacy” pairing with a 4-digit PIN can be cracked by sniffing. Modern **Secure Simple Pairing** and **LE Secure Connections** use elliptic curve Diffie–Hellman. Attacks like *BlueBorne* (2017) and *KNOB* (2019, which forced a 1-byte key) show that Bluetooth stacks must be patched. Turn Bluetooth off when not needed and do not accept unknown pairing requests.

8.7.2 Mobile phones

Phones are small computers that are always connected and often lost. Main risks:

- malicious apps (especially from outside the official stores) and too many app permissions;
- lost or stolen devices with no screen lock or encryption;
- phishing by SMS ("smishing") and fake apps;
- old devices that no longer get security updates.

Organizations use **Mobile Device Management (MDM)** to force a screen lock, encryption, updates and remote wipe.

8.7.3 Cellular networks

In **2G (GSM)**, only the phone proves its identity; the network does not. A fake base station ("IMSI catcher") can make phones connect to it and then turn off encryption. **3G and 4G** added mutual authentication (the AKA protocol). **5G** also encrypts the permanent subscriber identity (SUPI is sent as an encrypted SUCI) to stop simple IMSI catchers. Attackers can still force some phones down to 2G, so some phones now let users disable 2G.

Chapter summary

- Wireless signals can be heard and sent by anyone nearby: strong crypto is essential.
- WEP is broken; WPA2-AES is acceptable; WPA3 (SAE + PMF) is best.
- A captured WPA2-PSK handshake allows offline guessing: use long random passphrases.
- Evil twins and deauth attacks are stopped by certificate checks and PMF.
- Patch Bluetooth and phones; 5G protects the subscriber identity better than 2G.

Exercises

1. **[B]** List three reasons why WEP is insecure.
2. **[B]** What is the difference between WPA2-Personal and WPA2-Enterprise?
3. **[I]** With a 24-bit IV, how many packets are needed for a 50% chance of a repeated IV? Use the birthday formula $\approx 1.18\sqrt{2^{24}}$.
4. **[I]** A GPU tests 500 000 WPA2 passphrases per second. How long does it take to try all 8-digit numbers? All 10-character lowercase passwords?
5. **[E]** Explain why SAE in WPA3 stops offline dictionary attacks, while the WPA2 4-way handshake does not.
6. **[E]** Write a Python program that takes a "captured" PMK and SSID and tries a small word list to find the passphrase. Use it only on values you generate yourself.

CHAPTER 9

Malware and Application Attacks

What you will learn

- Know the main malware types and how they spread.
- Understand ransomware, botnets and advanced persistent threats.
- Understand how malware is detected and how to defend against it.
- Learn the top web attacks: SQL injection, XSS, CSRF and others.
- See SQL injection and XSS in Python, and fix them.

9.1 Malware types

Malware (malicious software) is any program written to harm a system or its users.

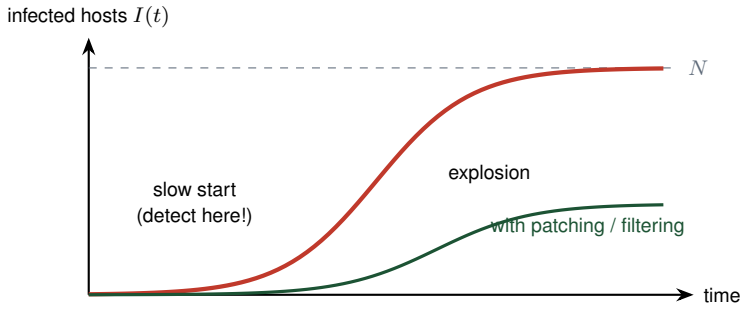
Type	What it does	How it spreads
Virus	Attaches itself to a host file or program; runs when the host runs.	Infected files, USB drives
Worm	A standalone program that copies itself over the network *without* user help.	Network vulnerabilities
Trojan	Looks useful (a free game, a cracked program) but hides a malicious part.	User downloads it
Ransomware	Encrypts files and asks for money for the key; often also steals data.	Phishing, RDP, VPN bugs
Spyware / **keylogger**	Records keys, screens, passwords and sends them out.	Trojans, bundled software
Rootkit	Hides itself and other malware deep in the OS or firmware.	After an attacker has admin rights
Bot	Lets a remote master control the machine as part of a botnet.	Worms, weak IoT passwords
Backdoor / **RAT**	Gives the attacker secret remote access.	Trojans, insiders

9.1.1 How a worm spreads

Worms can spread very fast. A simple model: each infected host scans random addresses and infects β new hosts per unit time while vulnerable hosts remain. This gives the **logistic curve**

$$\frac{dI}{dt} = \beta I \left(1 - \frac{I}{N}\right),$$

where I is the number of infected hosts and N is the number of vulnerable hosts. The growth is slow at first, explosive in the middle, then slows when most hosts are infected.



In 2003, the *SQL Slammer* worm infected most of its vulnerable targets in about 10 minutes. Human response is too slow; automatic defenses and patching *before* the attack are needed.

9.2 Ransomware

Ransomware is the biggest malware threat for organizations today. A modern attack:

1. **Initial access:** a phishing email, a stolen VPN or RDP password, or an unpatched Internet-facing server.
2. **Lateral movement:** the attacker steals admin passwords and moves through the network for days or weeks.
3. **Exfiltration:** copies sensitive data out (for "double extortion": pay or we publish).
4. **Destroy backups**, then **encrypt** all servers at the same time, often at night or on a holiday.

The best ransomware defense

Offline, tested backups. Follow the **3-2-1 rule**: 3 copies of data, on 2 different types of media, with 1 copy offline or off-site (and not reachable with normal admin passwords). Test that you can actually restore. Add MFA on remote access, fast patching and network segmentation.

9.3 Advanced persistent threats (APT)

An APT is a long, targeted campaign, usually by a nation state. "Advanced" means custom tools and zero-day exploits; "persistent" means the attacker stays hidden for months. **Stuxnet** (discovered 2010) is the famous example: it used several zero-days and spread by USB to reach industrial controllers that were not connected to the Internet, and physically damaged centrifuges. Lesson: an "air gap" is not a complete defense.

9.4 Detecting and stopping malware

- **Signature detection:** a database of known byte patterns or file hashes. Fast and accurate for known malware, but useless for new or changed malware. *Polymorphic* malware changes its code at each infection to avoid signatures.
- **Heuristic / behavior detection:** watches what a program *does*: encrypting many files quickly, injecting into other processes, turning off backups.
- **Sandboxing:** runs a suspicious file in a safe virtual machine and observes it.
- **EDR** (Endpoint Detection and Response): records activity on every computer, detects attacks and lets analysts isolate a machine with one click.
- **Application allow-listing:** only approved programs may run. Very strong, but hard to manage.

9.5 Social engineering

Many attacks start by fooling a person, not a computer.

- **Phishing:** fake emails that look real (bank, IT department, delivery company) asking the user to click a link, open a file or enter a password.
- **Spear phishing:** a phishing message written for one person, using real details about him.
- **Business email compromise:** a fake message from the “manager” asking for an urgent money transfer.
- **Pretexting and vishing:** a phone call: “I am from IT support, I need your password to fix your account.”

Defenses: user awareness training, email filtering, SPF/DKIM/DMARC to stop sender spoofing, MFA (a stolen password alone is not enough), and a simple way for users to report suspicious messages.

9.6 Web application attacks

Most services are now web applications, and web bugs are a very common way in. The **OWASP Top 10** lists the most important web risks, including broken access control, cryptographic failures, injection, insecure design and security misconfiguration.

9.6.1 SQL injection

If a program builds a database query by joining strings with user input, the user can change the meaning of the query.

How SQL injection works

The program builds:

```
SELECT * FROM users WHERE name='<input1>' AND password='<input2>'
```

The attacker types the password `' OR '1'='1`. The query becomes

```
SELECT * FROM users WHERE name='x' AND password='' OR '1'='1'
```

Because `AND` is evaluated before `OR`, the condition is `(false) OR true`, which is always true. The attacker logs in, and the query returns every user.

Lab — SQL injection and the fix `(c09_sqli.py)`

```
# Chapter 9: SQL injection -- the bug and the fix
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE users(name TEXT, password TEXT)")
db.executemany("INSERT INTO users VALUES (?, ?)",
               [("ali", "s3cret"), ("mona", "p@ss")])

def login_bad(name, pw):        # string building -> injectable
    q = f"SELECT * FROM users WHERE name='{name}' AND password='{pw}'"
    return db.execute(q).fetchall()

def login_good(name, pw):       # parameters -> data stays data
    q = "SELECT * FROM users WHERE name=? AND password=?"
    return db.execute(q, (name, pw)).fetchall()

attack = "' OR '1'='1"
print("bad  login:", login_bad("x", attack))
```

```
print("good login:", login_good("x", attack))
```

Output:

```
bad  login: [('ali', 's3cret'), ('mona', 'p@ss')]
good login: []
```

The fix: parameterized queries

Never build SQL by joining strings. Use **parameters** (placeholders like ?). The database receives the query and the data *separately*, so data can never become code. Add least privilege for the database account and input validation as extra layers.

9.6.2 Cross-site scripting (XSS)

In XSS, the attacker puts JavaScript into a page that *other* users will see, for example in a comment. The victim's browser runs it with the rights of the site: it can steal session cookies, change the page, or act as the user.

- **Stored XSS:** the script is saved in the database (comments, profiles).
- **Reflected XSS:** the script is in a link; the server copies it into the reply.
- **DOM XSS:** the bug is in the page's own JavaScript.

Lab — Escaping output stops XSS (c09_xss.py)

```
# Chapter 9: stop XSS by escaping output
import html

comment = '<script>steal(document.cookie)</script>'
print("Unsafe page:", f"<p>{comment}</p>")
print("Safe page  :", f"<p>{html.escape(comment)}</p>")
```

Output:

```
Unsafe page: <p><script>steal(document.cookie)</script></p>
Safe page  : <p>&lt;script&gt;steal(document.cookie)&lt;/script&gt;</p>
```

Defenses: escape all output for its context (HTML, attribute, JavaScript, URL), use modern frameworks that escape by default, set a **Content Security Policy (CSP)** header, and mark cookies `HttpOnly`.

9.6.3 Cross-site request forgery (CSRF)

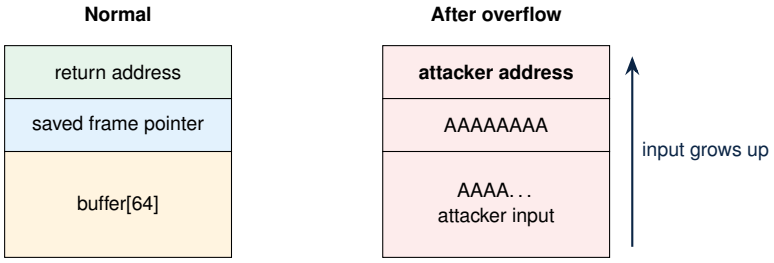
The victim is logged in to `bank.example`. He visits a bad site that contains a hidden form which sends "transfer 1000 to the attacker" to the bank. The browser automatically adds the bank cookie, so the bank thinks the user sent it. **Defenses:** anti-CSRF tokens in forms, the `SameSite` cookie attribute, and asking for the password again for critical actions.

9.6.4 Other important web bugs

Bug	Example	Defense
Broken access control	Changing `/invoice?id=1001` to `id=1002` shows another customer's invoice.	Check permission on the server for every request.
Command injection	Input `8.8.8.8; rm -rf /` given to a "ping" page.	Never pass input to a shell; use safe APIs.
Path traversal	`?file=../../etc/passwd`.	Allow-list file names; normalize paths.
SSRF	The server is tricked into fetching `http://169.254.169.254/` (cloud secrets).	Allow-list outgoing destinations.
Vulnerable components	An old library with a known bug (e.g. Log4Shell, 2021).	Keep an inventory of libraries; update fast.

9.6.5 Buffer overflow (a classic)

In languages like C, copying input into a fixed-size buffer without checking its length can overwrite nearby memory, including the *return address* on the stack. The attacker can then make the program jump to his own code.



Modern defenses: stack canaries, non-executable stack (NX/DEP), address space layout randomization (ASLR), and memory-safe languages (Rust, Go, Python, Java).

Chapter summary

- Viruses need a host, worms spread alone, trojans fool the user, ransomware encrypts and extorts.
- Offline tested backups, MFA, patching and segmentation are the core ransomware defenses.
- Signatures catch known malware; behavior analysis and EDR catch new malware.
- SQL injection: use parameterized queries. XSS: escape output and use CSP. CSRF: tokens and SameSite cookies.

Exercises

1. **[B]** What is the main difference between a virus and a worm?
2. **[B]** Write a short checklist for staff to recognize a phishing email.
3. **[I]** In the SQL lab, try the input `ali' --` as the user name with any password in `login_bad`. What happens? Why?
4. **[I]** Explain the 3-2-1 backup rule for a small college department.

5. **[E]** Use the logistic equation with $N = 100\,000$, $\beta = 1.5$ per hour and $I(0) = 1$. Solve it numerically in Python and find the time when half of the hosts are infected.
6. **[E]** Explain how ASLR makes a buffer overflow exploit harder, and why a memory leak bug can defeat it.

CHAPTER 10

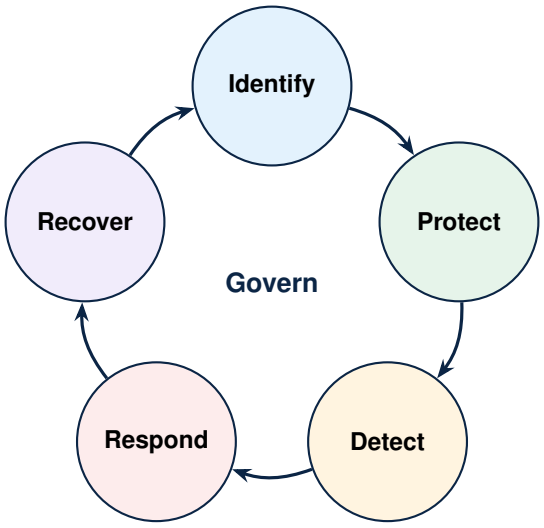
Security Management and Monitoring

What you will learn

- Understand logging, SIEM and the work of a security operations center (SOC).
- Do qualitative and quantitative risk assessment (ALE).
- Know the main standards and frameworks: ISO/IEC 27001 and NIST CSF.
- Manage vulnerabilities with CVE and CVSS, and know what a penetration test is.
- Follow the incident response life cycle and plan for business continuity.

So far we studied *technology*. But security fails most often because of *management*: no one reads the logs, patches wait for months, nobody knows what to do during an attack. This chapter is about running security as a process.

10.1 Security is a cycle



This is the **NIST Cybersecurity Framework (CSF 2.0)**. *Govern* sits in the center: leaders set the strategy, roles and policies. Then:

- **Identify:** know your assets, data and risks.
- **Protect:** access control, training, encryption, patching.
- **Detect:** logging, monitoring, IDS.
- **Respond:** handle incidents.
- **Recover:** restore services and learn lessons.

10.2 Security policies

A **policy** is a high-level, written statement from management: “All remote access must use MFA.” Below it come **standards** (“use WPA3 or WPA2-Enterprise”), **procedures** (step-by-step: “how to create a new user

account") and **guidelines** (advice).

Important policies for any organization:

- **Acceptable use policy:** what users may and may not do with IT resources.
- **Password and access policy**, including removing accounts when people leave.
- **Data classification:** public, internal, confidential, secret, with rules for each.
- **Backup, patch and incident response policies.**

10.3 Standards and frameworks

Name	What it is
ISO/IEC 27001	International standard for an Information Security Management System (ISMS). Organizations can be certified. Its Annex A lists 93 controls (2022 version).
NIST CSF 2.0	Free US framework (Govern, Identify, Protect, Detect, Respond, Recover). Very popular as a common language.
CIS Controls	A short, practical, prioritized list of 18 control groups. Good starting point for small organizations.
PCI DSS	Required for anyone who processes payment cards.

ISO 27001 follows the **Plan–Do–Check–Act** cycle: plan the controls, do (implement) them, check (audit and measure) them, and act (improve).

10.4 Risk assessment

10.4.1 Qualitative

We used the likelihood $\times$ impact matrix in Chapter 1. It is quick and easy to explain to management, but the numbers 1–5 are subjective.

10.4.2 Quantitative

Here we use money:

$$\text{SLE} = \text{Asset value (AV)} \times \text{Exposure factor (EF)} \qquad \text{(loss from one event)}$$

$$\text{ALE} = \text{SLE} \times \text{ARO} \qquad \text{(expected loss per year)}$$

where EF is the fraction of the asset lost in one event, and ARO is the annual rate of occurrence.

Is the control worth it?

A database is worth 2 000 000 EGP. A ransomware event would destroy 40% of its value (recovery cost, downtime). Such an event is expected once every 4 years.

- SLE $= 2\,000\,000 \times 0.4 = 800\,000$ EGP.
- ARO $= 1/4 = 0.25$, $\quad \text{ALE}_{\text{before}} = 800\,000 \times 0.25 = 200\,000$ EGP per year.

A backup and EDR solution costs 60 000 EGP per year and lowers the ARO to once in 20 years (0.05) and the EF to 10%:

- $\text{ALE}_{\text{after}} = 2\,000\,000 \times 0.1 \times 0.05 = 10\,000$ EGP per year.
- Value of the control $= 200\,000 - 10\,000 - 60\,000 = 130\,000$ EGP per year. **Buy it.**

10.5 Logging and monitoring

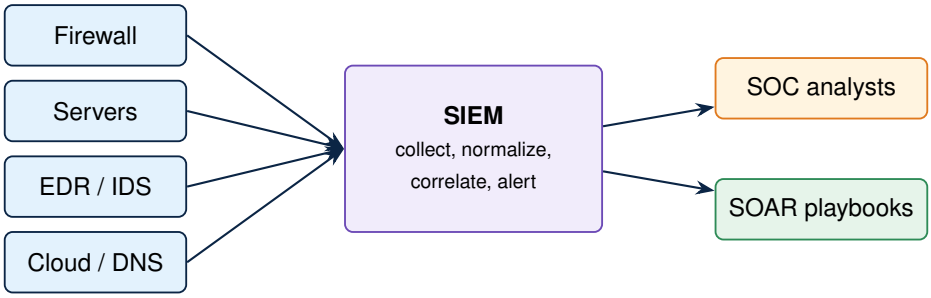
You cannot detect what you do not record. Important log sources:

- firewalls, VPN gateways and proxies (who connected where);
- authentication logs (Windows events, Linux `auth.log`, Active Directory);
- servers and applications (web access logs, database logs);
- endpoint (EDR) and IDS alerts; DNS queries; cloud audit logs.

Logs must have **correct time** on all devices (use NTP), must be sent to a central place the attacker cannot easily erase, and must be kept long enough (often months; attackers stay hidden a long time).

10.5.1 SIEM and SOC

A **SIEM** (Security Information and Event Management) collects logs from all sources, normalizes them, **correlates** them and raises alerts.



A correlation rule

One event alone is not alarming: “failed login” happens every day. But the rule

5 or more failed logins from one IP, followed by a successful login from the same IP within 10 minutes

very likely means a successful password-guessing attack. The SIEM correlates the events and raises a critical alert.

Lab — Find brute force in an auth log `(c10_logs.py)`

```
# Chapter 10: find brute-force logins in an auth log
import re
from collections import Counter

log = """\
Nov 08 10:00:01 sshd: Failed password for root from 45.9.1.2
Nov 08 10:00:02 sshd: Failed password for root from 45.9.1.2
Nov 08 10:00:03 sshd: Failed password for admin from 45.9.1.2
Nov 08 10:00:04 sshd: Failed password for root from 45.9.1.2
Nov 08 10:00:05 sshd: Failed password for root from 45.9.1.2
Nov 08 10:00:06 sshd: Failed password for root from 45.9.1.2
Nov 08 10:02:10 sshd: Failed password for sara from 192.168.1.30
Nov 08 10:02:15 sshd: Accepted password for sara from 192.168.1.30
Nov 08 10:05:00 sshd: Accepted password for root from 45.9.1.2"""

fails = Counter(re.findall(r"Failed password for \S+ from (\S+)", log))
ok = re.findall(r"Accepted password for (\S+) from (\S+)", log)
for ip, n in fails.items():
    if n >= 5:
        print(f"ALERT: {n} failed logins from {ip}")
for user, ip in ok:
```

```
    if fails[ip] >= 5:
        print(f"CRITICAL: {user} logged in from {ip} after brute force!")
```

Output:

```
ALERT: 6 failed logins from 45.9.1.2
CRITICAL: root logged in from 45.9.1.2 after brute force!
```

A **Security Operations Center (SOC)** is the team that watches the SIEM 24/7, triages alerts and starts incident response. **SOAR** tools automate common steps (for example: block the IP on the firewall and disable the account).

10.6 Vulnerability management

New vulnerabilities are published every day. Each gets an ID from the **CVE** system, for example CVE-2021-44228 (Log4Shell). Its severity is scored with **CVSS** from 0 to 10:

CVSS score	Rating	Typical response time
9.0–10.0	**Critical**	days (or hours if exploited)
7.0–8.9	**High**	2–4 weeks
4.0–6.9	Medium	next patch cycle
0.1–3.9	Low	when possible

The process: **(1)** keep an inventory of all assets and software, **(2)** scan regularly with a vulnerability scanner, **(3)** prioritize (CVSS + is it exploited in the wild? + is the asset exposed to the Internet?), **(4)** patch or mitigate, **(5)** verify.

Key idea

Attackers often use vulnerabilities that are *already public and patched*. The time between patch release and attack is now very short for important bugs. Fast patching of Internet-facing systems is one of the most effective controls you can have.

10.6.1 Penetration testing

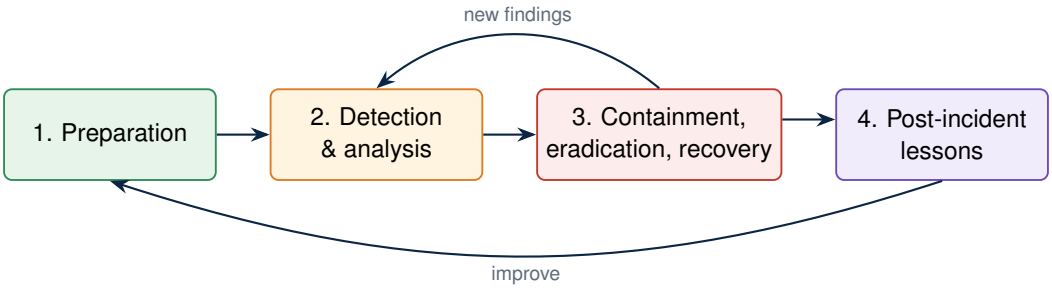
A **penetration test** is an authorized, simulated attack done by professionals to find weaknesses before real attackers do. It always starts with a written agreement on **scope** and **rules of engagement**.

- **Black box:** the testers know nothing, like an outside attacker.
- **White box:** the testers get full information and source code.
- **Grey box:** something in between, for example a normal user account.

A **red team** plays the attacker over a longer time; the **blue team** defends; a **purple team** exercise has them work together to improve detection.

10.7 Incident response

An incident is an event that harms, or may harm, the CIA of systems. The NIST incident response life cycle has four phases:



1. **Preparation:** a written plan, a team with clear roles, contact lists (management, legal, ISP, national CERT), tools and offline backups. Practice with table-top exercises.
2. **Detection and analysis:** confirm it is a real incident, find its scope and severity, keep notes with times.
3. **Containment, eradication, recovery:** isolate infected machines (but do not just switch them off: memory evidence is lost), remove the malware and the attacker's access, reset passwords, restore from clean backups, watch closely.
4. **Lessons learned:** within two weeks, meet and answer: what happened, what worked, what must change?

Digital evidence

If the incident may go to court, keep a **chain of custody**: who collected each piece of evidence, when, and where it was kept. Work on copies (disk images with hashes), never on the original.

10.8 Business continuity and disaster recovery

- **BCP** (business continuity plan): how the organization keeps working during a disaster.
- **DRP** (disaster recovery plan): how IT systems are restored.
- **RTO** (recovery time objective): the maximum time a service may be down.
- **RPO** (recovery point objective): the maximum data loss, measured in time. If RPO is 4 hours, backups must run at least every 4 hours.

Recovery sites: a **hot site** (ready in minutes, expensive), a **warm site** (hours to days) and a **cold site** (only space and power, weeks).

Chapter summary

- NIST CSF: Govern, Identify, Protect, Detect, Respond, Recover. ISO 27001 builds an ISMS.
- ALE $=$ AV $\times$ EF $\times$ ARO helps decide if a control is worth its cost.
- Central logging + SIEM correlation + a SOC turn raw events into detection.
- Patch by risk (CVSS, exploitation, exposure); test with authorized pentests.
- Prepare for incidents before they happen; define RTO and RPO.

Exercises

1. **[B]** Put these in order from general to specific: procedure, policy, standard.
2. **[B]** What is the difference between RTO and RPO? Give an example for a student registration system.

3. **[I]** A laptop is worth 40 000 EGP; EF $= 1$; 5 laptops are stolen per year. Compute the ALE. Is a 150 000 EGP per year tracking and encryption system worth it if it halves thefts?
4. **[I]** Extend the log lab to also detect “password spraying”: one IP that fails on many *different* user names.
5. **[E]** Write an incident response plan (one page) for a ransomware attack on the college file server.
6. **[E]** Compare ISO 27001 and NIST CSF: who uses each, is certification possible, and how are they structured?

CHAPTER 11

Emerging Topics

What you will learn

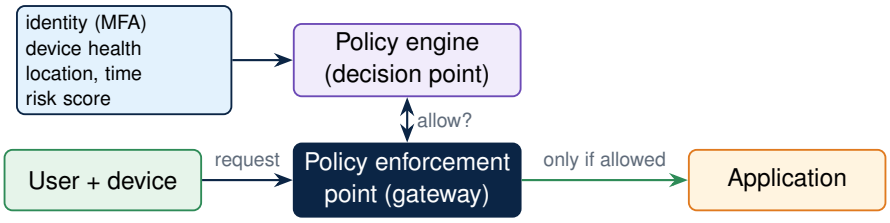
- Understand the zero trust model and build a small policy engine.
- Know the shared responsibility model of cloud security.
- Understand the special problems of IoT and industrial (OT) networks.
- Understand the quantum threat and post-quantum cryptography; sign with a hash-based signature.
- See how AI is used by both attackers and defenders, and plan your path to expert.

11.1 Zero trust

The old model was the **castle and moat**: strong walls (firewall) and full trust for everything inside. But today users work from home, data is in the cloud, and attackers who get one password are "inside". The perimeter has dissolved.

Zero trust in one sentence

Never trust, always verify. No user, device or network is trusted by default; *every* request is authenticated, authorized and encrypted, using as much context as possible, and gets only the least privilege it needs.



The main ideas (NIST SP 800-207):

- Strong identity for users *and* devices (MFA, device certificates).
- Check device health (patched? disk encrypted? EDR running?).
- **Micro-segmentation:** each application is its own small zone.
- Decide per request, continuously, not once per day.
- Assume breach: log and monitor everything.

Lab — A tiny zero-trust policy engine `(c11_zerotrust.py)`

```
# Chapter 11: a tiny zero-trust policy check (every request is checked)
POLICY = {
    "grades-app": {"roles": {"teacher"}, "mfa": True, "device_ok": True},
    "library":    {"roles": {"student", "teacher"}, "mfa": False, "device_ok": False},
}

def allow(req):
```

```
    p = POLICY[req["app"]]
    reasons = []
    if req["role"] not in p["roles"]:       reasons.append("role")
    if p["mfa"] and not req["mfa"]:         reasons.append("no MFA")
    if p["device_ok"] and not req["device"]: reasons.append("bad device")
    if req["risk"] > 70:                    reasons.append("high risk")
    return ("ALLOW", []) if not reasons else ("DENY", reasons)

for r in [
    {"user":"dr.hany","role":"teacher","app":"grades-app","mfa":True,"device":True,"risk":10},
    {"user":"dr.hany","role":"teacher","app":"grades-app","mfa":True,"device":False,"risk":10},
    {"user":"omar","role":"student","app":"grades-app","mfa":True,"device":True,"risk":5},
    {"user":"omar","role":"student","app":"library","mfa":False,"device":False,"risk":20},
]:
    print(r["user"], "->", r["app"], ":", *allow(r))
```

Output:

```
dr.hany -> grades-app : ALLOW []
dr.hany -> grades-app : DENY ['bad device']
omar -> grades-app : DENY ['role']
omar -> library : ALLOW []
```

11.2 Cloud security

In the cloud, someone else runs the hardware. Who is responsible for what? This is the **shared responsibility model**.



The customer is *always* responsible for its data and for who can access it.

Most cloud breaches are configuration mistakes

Typical causes: storage buckets left public, access keys pushed to GitHub, admin accounts without MFA, too many permissions, and no logging. Use the provider's security tools (posture management), least-privilege IAM roles, MFA for every console account, and encryption with keys you control.

11.3 IoT and industrial networks

11.3.1 Internet of Things

Cameras, smart TVs, sensors and medical devices are small computers with weak security: default passwords, no updates, old software, and no screen to show warnings. The Mirai botnet (Chapter 1) used only a list of about 60 default user names and passwords.

Defenses: change default passwords, put IoT devices in a separate VLAN with strict firewall rules, block their access to the Internet if not needed, and buy devices from vendors that provide updates.

11.3.2 Operational technology (OT / ICS / SCADA)

OT networks control physical processes: power grids, water plants, factories. Their priorities are different:

	IT network	OT network
Most important	Confidentiality	**Availability and safety**
Device life	3–5 years	15–30 years
Patching	monthly	rare (needs plant shutdown)
Protocols	TLS, modern	Modbus, DNP3 (often no authentication)

Modbus, for example, has no authentication at all: anyone who can reach the device can send "write register" commands. Defenses follow the **Purdue model**: separate OT from IT in layers, with a DMZ between them, one-way gateways (data diodes) where possible, and OT-aware monitoring. The standard for this area is **IEC 62443**.

11.4 Quantum computing and post-quantum cryptography

11.4.1 The threat

A large, error-corrected quantum computer would change cryptography:

- **Shor's algorithm** factors integers and solves discrete logarithms in polynomial time. This **breaks RSA, Diffie–Hellman and ECC** completely.
- **Grover's algorithm** searches N items in about $\sqrt{N}$ steps. A 128-bit key gives only about 64-bit security against it, so we use **AES-256**. Hashes need larger outputs for the same reason. Symmetric crypto survives.

Algorithm	Classical security	Against a quantum computer
RSA-2048, ECC P-256, DH	strong	**broken** (Shor)
AES-128	128 bits	about 64 bits (Grover)
AES-256	256 bits	about 128 bits: **safe**
SHA-256 / SHA-384	strong	still safe for most uses

Harvest now, decrypt later

Nobody has such a quantum computer today. But an attacker can **record** encrypted traffic now and decrypt it in 10–15 years. Data that must stay secret for a long time (military, medical, state secrets) is already at risk. This is why migration has started now.

11.4.2 The new standards

In August 2024, NIST published the first post-quantum (PQC) standards. They run on normal computers:

Standard	Old name	Use and basis
FIPS 203: ML-KEM	Kyber	Key exchange / encapsulation. Lattices (Module-LWE).
FIPS 204: ML-DSA	Dilithium	Signatures. Lattices.
FIPS 205: SLH-DSA	SPHINCS+	Signatures. Only hash functions: very conservative.

More algorithms are coming (for example FN-DSA based on Falcon, and HQC, a code-based KEM selected in 2025 as a backup to ML-KEM).

Today, the common approach is **hybrid** key exchange: combine classical X25519 with ML-KEM, so the connection is safe if *either* one is safe. Major browsers and several large web services have already enabled hybrid ML-KEM in TLS 1.3.

11.4.3 Hash-based signatures: Lamport

The simplest post-quantum signature uses only a hash function (Lamport, 1979):

1. **Key generation:** pick 256 pairs of random secrets $(s_{i,0}, s_{i,1})$. The public key is their hashes $(H(s_{i,0}), H(s_{i,1}))$.
2. **Sign** message m: compute $h = H(m)$ (256 bits). For each bit b_i of h, reveal the secret s_{i,b_i}.
3. **Verify:** hash each revealed secret and compare with the public key part chosen by the bit.

Forging needs inverting the hash, which even a quantum computer cannot do efficiently. The key can be used **only once**: a second signature reveals more secrets. SLH-DSA builds on this idea with trees of one-time keys.

Lab — Lamport one-time signature `(c11_lamport.py)`

```python
# Chapter 11: Lamport one-time signature -- safe against quantum computers
import hashlib, os

H = lambda b: hashlib.sha256(b).digest()

def keygen():
    sk = [[os.urandom(32), os.urandom(32)] for _ in range(256)]
    pk = [[H(a), H(b)] for a, b in sk]
    return sk, pk

def bits(msg):
    d = int.from_bytes(H(msg), "big")
    return [(d >> (255 - i)) & 1 for i in range(256)]

def sign(sk, msg):
    return [sk[i][b] for i, b in enumerate(bits(msg))]

def verify(pk, msg, sig):
    return all(H(s) == pk[i][b] for i, (s, b) in enumerate(zip(sig, bits(msg))))

sk, pk = keygen()
sig = sign(sk, b"hello")
print("public key size:", 256 * 2 * 32, "bytes")
print("valid  :", verify(pk, b"hello", sig))
print("changed:", verify(pk, b"hellO", sig))
```

Output:

```
public key size: 16384 bytes
valid  : True
changed: False
```

Quantum key distribution (QKD)

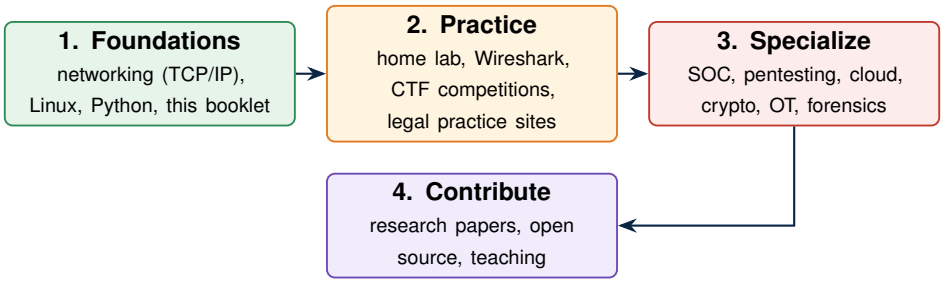
QKD (for example the BB84 protocol) uses single photons to share a key. Its security comes from physics: measuring a quantum state disturbs it, so an eavesdropper creates errors that the two sides can detect. QKD needs special hardware and an authenticated classical channel, and real devices have had implementation attacks. Many agencies currently recommend PQC as the main migration path, with QKD for special links.

11.5 AI and security

- **AI for defense:** anomaly detection in network traffic, malware classification, alert triage in the SOC, summarizing incidents, finding bugs in code.
- **AI for attack:** more convincing phishing in any language, deepfake voices and videos for fraud ("the CEO calls and asks for a transfer"), faster discovery of vulnerabilities.
- **Attacks on AI systems:** adversarial examples (small changes that fool a classifier), data poisoning of training sets, and **prompt injection** against chatbots and AI agents (hidden instructions in a web page or document).

Defense against deepfake fraud is mostly a process: verify unusual requests through a second, known channel.

11.6 Your path to expert



Well-known certifications: CompTIA Security+ (entry level), CEH and OSCP (penetration testing), CISSP (management, needs experience), and vendor certifications (Cisco, cloud providers). Certificates help, but practice and curiosity make an expert.

Chapter summary

- Zero trust: verify every request with identity, device and context; least privilege.
- In the cloud, the customer always owns data and access security; most breaches are misconfigurations.
- IoT and OT need segmentation; in OT, availability and safety come first.
- Shor's algorithm breaks RSA/ECC; migrate to ML-KEM, ML-DSA and SLH-DSA; use AES-256.
- AI helps both sides; verify unusual requests through a second channel.

Exercises

1. [B] Explain "never trust, always verify" with an example from your college.
2. [B] In IaaS, who patches the operating system? In SaaS?
3. [I] Add a "time of day" rule to the zero-trust lab: the grades app is allowed only from 08:00 to 18:00.
4. [I] Why does Grover's algorithm force us to double symmetric key sizes, while Shor's algorithm forces us to replace RSA completely?
5. [E] In the Lamport lab, sign two different messages with the same key. Count how many secret pairs are now fully revealed, and explain how a forger can use them.
6. [E] Draw the Purdue model for a small water treatment plant, and place firewalls and a DMZ.

CHAPTER 12

Capstone: Securing a Campus Network

What you will learn

- Use everything from Chapters 1–11 on one realistic network.
- Go from risk assessment to design, to detection, to incident response.
- Test yourself with a final self-test.

12.1 The scenario

A college has 3 000 students, 250 staff and two sites (main campus and a branch 50 km away). Today the network is "flat": every device is in one big LAN, Wi-Fi uses one WPA2 password written on the wall, the student portal runs on an old server, backups go to a USB disk that is always connected, and the admins use one shared "admin" account. Last month a staff member received a phishing email and gave his password.

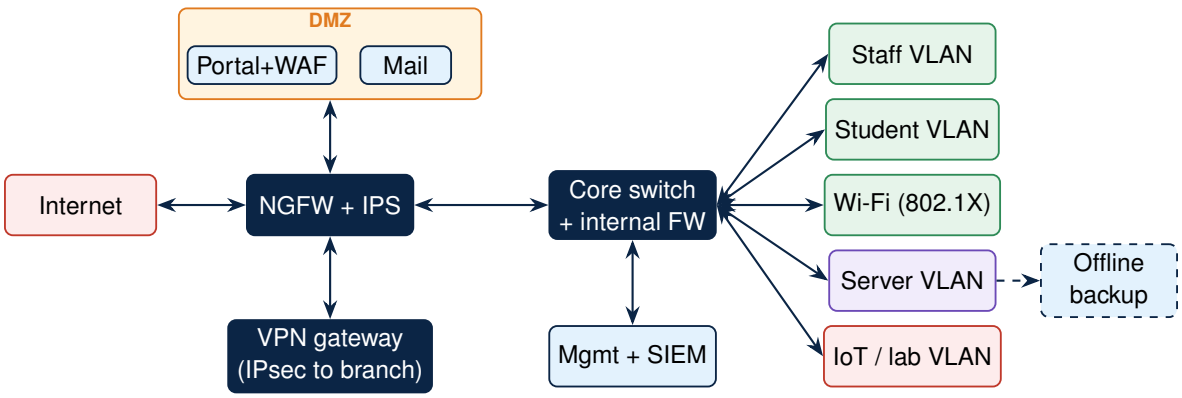
Your job: make a security plan.

12.2 Step 1: Identify assets and risks

Asset	L	I	Main threat
Student records and grades	4	5	insider change, ransomware, data theft
Student portal (web)	5	4	SQL injection, XSS, DDoS
Staff email accounts	5	4	phishing, password reuse
Wi-Fi network	4	3	shared password, evil twin
Backups	3	5	encrypted together with servers
Lab and IoT devices	4	2	default passwords, botnets

(L = likelihood, I = impact, from 1 to 5.) Rank them with the Chapter 1 lab: five of the six are HIGH (score 15 or more).

12.3 Step 2: Design the new network



1. **Segmentation (Ch. 6):** separate VLANs for staff, students, Wi-Fi, servers, IoT and management. Default deny between them; only the needed ports are open.

2. **DMZ (Ch. 6):** the student portal and mail server move to the DMZ, behind a WAF.
3. **Secure coding (Ch. 9):** review the portal for SQL injection and XSS; use parameterized queries and output escaping; set CSP and secure cookies.
4. **TLS everywhere (Ch. 5):** HTTPS with TLS 1.2/1.3 only, HSTS, valid certificates.
5. **Branch link (Ch. 7):** site-to-site IPsec (IKEv2, AES-GCM) between the two campuses.
6. **Wi-Fi (Ch. 8):** WPA3/WPA2-Enterprise with 802.1X and personal accounts; certificate checking on clients; guest network separate; PMF on.

12.4 Step 3: Identity and access

- MFA for all staff, all remote access and all admins (Ch. 3); passkeys or FIDO2 keys for admins.
- Remove the shared "admin" account. Each admin gets a personal admin account, used only for admin work (least privilege, accounting).
- RBAC for the student system: student, teacher, registrar, admin roles. Grades can be changed only by the teacher of the course, and every change is logged.
- Passwords stored with Argon2id or bcrypt on all in-house systems.

12.5 Step 4: Detect

- Central logging to the SIEM from the firewall, VPN, Wi-Fi controller, Active Directory, portal and servers (Ch. 10). NTP on all devices.
- Correlation rules: brute force followed by success; logins from two countries within one hour; grade changes outside work hours; large data transfers to the Internet.
- IPS on the NGFW and EDR on all staff PCs and servers.
- A honeytoken: a fake "admin_backup" account that nobody uses. Any login attempt is an alarm.

12.6 Step 5: Respond and recover

- Backups follow 3-2-1: daily backups, one copy offline and one off-site; a restore test every term. Set RPO $=$ 24 hours and RTO $=$ 8 hours for the student system.
- A written incident response plan with roles and phone numbers; a table-top exercise for a ransomware case every year.
- Phishing awareness training for staff each term, and a "report phishing" button in email.

12.7 Step 6: Prepare for the future

- Move step by step to zero trust: application access through an identity-aware gateway instead of full network VPN for staff (Ch. 11).
- Start a cryptographic inventory: which systems use RSA/ECC and for how long their data must stay secret. Plan the move to hybrid ML-KEM when vendors support it.

Key idea

Notice that no single product solves the problem. The plan uses **people** (training, roles), **process** (policies, incident response, patching) and **technology** (firewalls, crypto, MFA, SIEM) together. This is defense in depth from Chapter 1, now at full scale.

12.8 Final self-test

Answer without looking back. Answers are at the end of this chapter.

1. Which CIA goal does a DDoS attack target?
2. Why is AES-ECB not used for long messages?
3. In RSA, which key is used to verify a signature?
4. What does a salt protect against in password storage?
5. Name the two values TOTP feeds into HMAC.
6. What attack sends fake ARP replies?
7. What is the main defense of a server against a SYN flood?
8. Which TLS 1.3 property protects old traffic if the server key is stolen?
9. In IPsec, which mode hides the original IP addresses?
10. In a firewall rule list, which rule is applied to a packet?
11. Why is a DMZ used?
12. What is the main weakness of signature-based IDS?
13. What is the typical overhead of an IPsec ESP tunnel per packet?
14. Why is a captured WPA2-PSK handshake dangerous?
15. Which WPA3 feature stops deauthentication attacks?
16. What is the difference between a virus and a worm?
17. How do you fix SQL injection?
18. What does the 3-2-1 backup rule say?
19. Write the formula for ALE.
20. What are the four phases of NIST incident response?
21. State the zero trust principle in four words.
22. Which quantum algorithm breaks RSA and ECC?
23. Name the NIST post-quantum standard for key encapsulation.
24. Why must a Lamport key be used only once?
25. In the cloud (IaaS), who is responsible for the security of the data?

Answers to the self-test

1. Availability. 2. Equal blocks give equal ciphertext, so patterns leak. 3. The public key. 4. Rainbow tables and same-password detection across users. 5. The shared secret and the time counter $\lfloor t/30 \rfloor$. 6. ARP poisoning (spoofing). 7. SYN cookies. 8. Forward secrecy. 9. Tunnel mode. 10. The first matching rule. 11. To keep public servers apart from the internal network. 12. It cannot detect new (zero-day) attacks. 13. About 60 bytes. 14. It allows offline password guessing. 15. Protected Management Frames (PMF). 16. A virus needs a host file and user action; a worm spreads by itself. 17. Parameterized queries. 18. 3 copies, 2 media types, 1 offline / off-site. 19. ALE $=$ AV $\times$ EF $\times$ ARO. 20. Preparation; detection and analysis; containment, eradication and recovery; post-incident lessons. 21. Never trust, always verify. 22. Shor's algorithm. 23. ML-KEM (FIPS 203). 24. Each signature reveals half of the secrets; two signatures reveal enough to forge. 25. The customer.

Your score: 22–25: expert level. 16–21: very good, review the chapters of your mistakes. Below 16: read Chapters 1–6 again and repeat the labs.

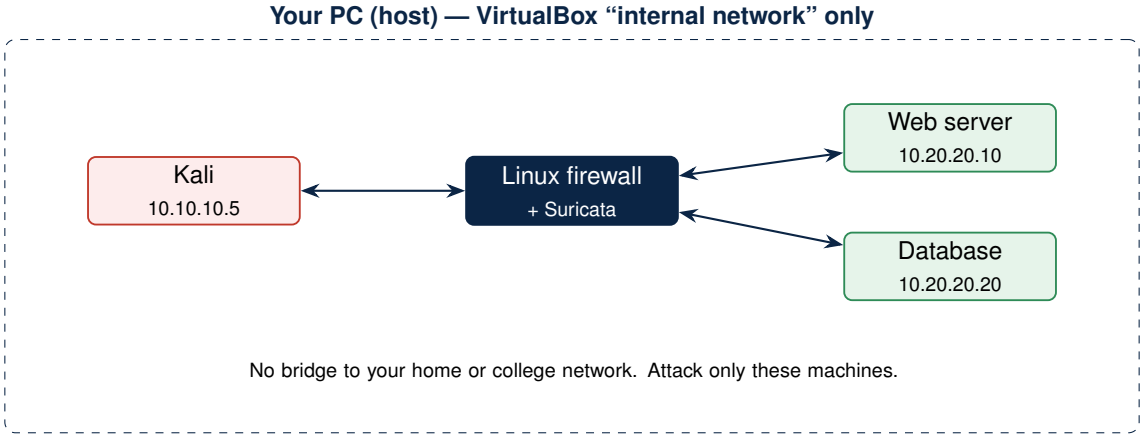
APPENDIX A

Building Your Practice Lab

A.1 Software you need

Tool	Use in this booklet	Cost
Python 3 + `cryptography`	All labs	Free
VirtualBox or VMware Player	Run several virtual machines on one PC	Free
Linux VM (Ubuntu, Debian)	Servers, firewalls (iptables / nftables), SSH	Free
Kali Linux VM	Security tools collection (Nmap, Wireshark, . . .)	Free
Wireshark	See packets, protocols and handshakes	Free
Nmap	Scan *your own* lab machines	Free
Suricata or Snort	Network IDS	Free
WireGuard / strongSwan	VPN labs	Free

A.2 A safe home lab design



A.3 Suggested lab sessions

1. **Wireshark basics:** capture a plain HTTP login and an HTTPS login on your lab web server. Compare what you can see.
2. **TLS handshake:** open a TLS 1.3 connection and find ClientHello, ServerHello and the key share in Wireshark.
3. **Scanning:** scan the web server with Nmap, then close the unused ports and scan again.
4. **Firewall:** write nftables or iptables rules for the DMZ design of Chapter 6 and test them.
5. **IDS:** write a Suricata rule for the SQL injection pattern and trigger it from Kali.
6. **SSH hardening:** use key login, disable passwords and root login, and install fail2ban.
7. **VPN:** connect two VMs with WireGuard and check the MTU with `ping -s`.
8. **Logging:** send all logs to one VM and run the Chapter 10 log lab on them.

Be careful

Keep the lab on a host-only or internal virtual network. Tools that scan or attack must never leave the lab. If you want more practice, use legal training platforms and capture-the-flag (CTF) competitions that clearly allow it.

APPENDIX B

Answers to Selected Exercises

Chapter 1

1. (a) Confidentiality, (b) integrity, (c) availability.

3. A passive attacker changes nothing, so there is nothing to see in logs or traffic; we cannot detect him, so we must make what he hears useless (encryption). An active attacker changes or sends traffic, which leaves traces that integrity checks and monitoring can detect.

Chapter 2

1. `KHOOR` $\rightarrow$ `HELLO`.

3. $n = 187$, $\varphi(n) = 160$, $d = 23$ (since $7 \times 23 = 161 = 160 + 1$), $c = 88^7 \bmod 187 = 11$.

4. $A = 5^4 \bmod 23 = 4$, $B = 5^3 \bmod 23 = 10$, $K = 10^4 \bmod 23 = 4^3 \bmod 23 = 18$.

6. In CTR, $C_i = P_i \oplus S$ where S is the key stream from (key, nonce). Same nonce $\Rightarrow$ same S, so $C_1 \oplus C_2 = P_1 \oplus P_2$. If one plaintext is known or guessed, the other is revealed.

7. 23 people.

Chapter 3

1. No. Both are “something you know”; one phishing page can steal both.

2. $10 \log_2 10 \approx 33.2$ bits.

5. Write: yes (“write up” is allowed; only write down is forbidden). Read: no (no read up).

Chapter 4

1. ARP poisoning: link layer. SYN flood: transport. DNS spoofing: application.

2. The server’s SYN+ACK goes to the spoofed address, so the attacker never learns the server’s random sequence number y and cannot send the correct ACK.

4. Factor $= 3000/60 = 50$. The attacker must send $10/50 = 0.2$ Gbit/s $= 200$ Mbit/s, so 200 bots of 1 Mbit/s.

5. The server’s sequence number is a keyed hash of the connection data and time. When the ACK comes back with $y + 1$, the server recomputes the hash and checks it, so no table entry was needed. Some TCP options (like large window scaling values) can be lost because there is no room to store them.

Chapter 5

1. Confidentiality, integrity, server authentication. Without the certificate check, authentication is missing, and a MITM can read everything.

3. With forward secrecy, each 2026 session used its own temporary DH keys, which were deleted. The stolen 2027 private key was only used for signing, so it cannot recover the old session keys.

6. AH protects the outer IP header, including the addresses. NAT changes the address, so the integrity check fails. ESP does not protect the outer header, and with UDP encapsulation (port 4500) NAT can change ports and addresses freely.

Chapter 6

4. Detection rate $= 45/50 = 90\%$. False alarm rate $= 200/99\,950 \approx 0.2\%$.

6. False alarms: $0.0001 \times 1\,000\,000 = 100$; true alerts ≈ 10. So about $10/110 \approx 9\%$ of alerts are real: much better, but still most alerts are false.

Chapter 7

3. $1492 - 73 = 1419$ bytes.

5. The header (SPI + sequence) is passed to AES-GCM as associated data. Changing it makes the tag check fail (`InvalidTag`), so the packet is rejected. This is why the header is authenticated even though it is not encrypted.

Chapter 8

3. $1.18 \times \sqrt{2^{24}} = 1.18 \times 4096 \approx 4833$ packets.

4. 8 digits: $10^8/500\,000 = 200$ seconds. 10 lowercase letters: $26^{10}/500\,000 \approx 2.8 \times 10^8$ s ≈ 9 years.

5. In WPA2, the captured handshake contains a MIC that depends only on the password and public values, so guesses can be checked offline. In SAE, each guess needs a fresh exchange with the access point (the transcript does not let the attacker test guesses), so the attacker is limited to slow, visible online guessing.

Chapter 9

3. The query becomes `... WHERE name='ali' -' AND password='...'`. The `-` starts a comment, so the password check is removed and the attacker logs in as `ali`. (Tested: `login_bad` returns ali's row.)

5. The solution is $I(t) = N/(1+(N-1)e^{-\beta t})$. Half infected when $e^{\beta t} = N-1$, so $t = \ln(99\,999)/1.5 \approx 7.7$ hours.

Chapter 10

1. Policy $\rightarrow$ standard $\rightarrow$ procedure.

3. ALE $= 40\,000 \times 1 \times 5 = 200\,000$ EGP. Halving thefts saves $100\,000$ EGP per year, less than the $150\,000$ cost, so it is not worth it *on hardware value alone*. But if the laptops contain sensitive data, the real asset value is much higher, and encryption becomes clearly worth it.

Chapter 11

2. IaaS: the customer. SaaS: the provider.

4. Grover gives only a square-root speedup, so doubling the key length restores the security level. Shor solves the hard problem itself in polynomial time, so no reasonable key size helps.

5. For each of the 256 positions where the two message hashes have different bits (about 128), both secrets are revealed. A forger can then sign any new message whose hash bits match the revealed secrets at every position, and with many revealed pairs this becomes feasible.

APPENDIX C

Quick Reference

C.1 Common ports

Port	Proto	Service	Secure?	Port	Proto	Service	Secure?
20/21	TCP	FTP	no	443	TCP/UDP	HTTPS / QUIC	yes
22	TCP	SSH / SFTP	yes	445	TCP	SMB (file sharing)	internal only
23	TCP	Telnet	no	500/4500	UDP	IKE / IPsec NAT-T	yes
25	TCP	SMTP	with STARTTLS	636	TCP	LDAPS	yes
53	UDP/TCP	DNS	DNSSEC / DoT	853	TCP	DNS over TLS	yes
67/68	UDP	DHCP	no	1194	UDP	OpenVPN	yes
80	TCP	HTTP	no	1812/1813	UDP	RADIUS	partly
88	TCP/UDP	Kerberos	yes	3389	TCP	RDP	never open to Internet
123	UDP	NTP	partly	502	TCP	Modbus	no
161	UDP	SNMP v1/v2	no (use v3)	51820	UDP	WireGuard	yes

C.2 Which algorithm should I use? (2026)

Job	Use	Avoid
Encrypt data	AES-256-GCM, ChaCha20-Poly1305	DES, 3DES, RC4, AES-ECB
Hash	SHA-256, SHA-384, SHA-3, BLAKE2	MD5, SHA-1
MAC	HMAC-SHA256, Poly1305 (in AEAD)	homemade “hash(key+msg)”
Passwords	Argon2id, scrypt, bcrypt, PBKDF2 (many rounds)	plain SHA-256, MD5, encryption
Key exchange	X25519, hybrid X25519 + ML-KEM	static RSA key exchange, small DH groups
Signatures	Ed25519, ECDSA P-256, RSA-PSS 3072, ML-DSA	RSA PKCS#1 v1.5 for new designs, DSA
Protocols	TLS 1.3, SSH, IKEv2, WireGuard, WPA3	SSL, TLS 1.0/1.1, Telnet, WEP, PPTP
Random numbers	`secrets`, `os.urandom`	`random`, time-based seeds

C.3 Top 10 actions for any organization

1. Know your assets: a complete inventory of devices, software and data.
2. MFA for all remote access, email and admin accounts.
3. Patch Internet-facing systems fast; patch everything else regularly.
4. Offline, tested backups (3-2-1).
5. Least privilege: separate admin accounts; no daily work as admin.
6. Segment the network; put public servers in a DMZ.
7. Central logging with correct time; someone must look at alerts.
8. EDR on all endpoints.
9. Train users to recognize phishing and to report it.
10. Write and practice an incident response plan.

APPENDIX D

Glossary

AAA Authentication, Authorization, Accounting.

ACL Access Control List: the list of who may access an object, or a router's filter rules.

AEAD Authenticated Encryption with Associated Data, e.g. AES-GCM.

AES Advanced Encryption Standard, the main symmetric block cipher.

ALE Annualized Loss Expectancy $=$ SLE $\times$ ARO.

APT Advanced Persistent Threat: long, targeted, skilled attack campaign.

ARP poisoning Sending fake ARP replies to redirect LAN traffic.

Botnet Many infected machines controlled by one attacker.

CA Certificate Authority: signs certificates.

CIA Confidentiality, Integrity, Availability.

CSRF Cross-Site Request Forgery: making a logged-in browser send an unwanted request.

CVE / **CVSS** Public ID of a vulnerability / its severity score (0–10).

DDoS Distributed Denial of Service.

DMZ A network zone for public servers, between the Internet and the inside network.

DNSSEC Signed DNS answers.

ECC Elliptic Curve Cryptography.

EDR Endpoint Detection and Response.

ESP Encapsulating Security Payload, the IPsec protocol for encryption.

Exploit Code or a method that uses a vulnerability.

Forward secrecy Old sessions stay safe even if a long-term key is stolen later.

HMAC Hash-based Message Authentication Code.

IDS / **IPS** Intrusion Detection / Prevention System.

IKE Internet Key Exchange, sets up IPsec keys.

Kerberos Ticket-based single sign-on protocol.

MAC (crypto) Message Authentication Code (not the MAC address!).

MFA Multi-Factor Authentication.

MITM Man-in-the-middle attack.

ML-KEM / **ML-DSA** NIST post-quantum key exchange / signature standards.

Nonce A number used only once.

OTP / **TOTP** One-time password / time-based one-time password.

PKI Public Key Infrastructure.

PMK / **PTK** Pairwise Master / Transient Key in Wi-Fi security.

Ransomware Malware that encrypts data and asks for payment.

RBAC Role-Based Access Control.

Rootkit Malware that hides itself deep in the system.

RTO / **RPO** Recovery Time / Recovery Point Objective.

Salt Random value added to a password before hashing.

SIEM Security Information and Event Management.

SOC Security Operations Center.

Spoofing Faking an identity (IP address, email sender, caller).

SQL injection Changing a database query through user input.

TLS Transport Layer Security, the protocol behind HTTPS.

VPN Virtual Private Network.

WPA2 / **WPA3** Wi-Fi security standards.

XSS Cross-Site Scripting: injecting script into pages other users see.

Zero-day A vulnerability unknown to the vendor, with no patch yet.

Zero trust Security model: never trust, always verify every request.

APPENDIX E

Further Reading

- W. Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson. The classic textbook for this course.
- W. Stallings and L. Brown, *Computer Security: Principles and Practice*, Pearson.
- C. Kaufman, R. Perlman, M. Speciner and R. Perlner, *Network Security: Private Communication in a Public World*, Addison-Wesley.
- R. Anderson, *Security Engineering*, Wiley (3rd ed.). Earlier editions are free online from the author.
- J.-P. Aumasson, *Serious Cryptography*, No Starch Press. Modern, practical cryptography.
- D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*. Free online; for the expert level.
- NIST publications: Cybersecurity Framework 2.0; SP 800-61 (incident handling); SP 800-207 (zero trust); FIPS 203, 204, 205 (post-quantum).
- OWASP Top 10 and OWASP Cheat Sheet Series (owasp.org), for web security.
- RFC 8446 (TLS 1.3), RFC 7296 (IKEv2), RFC 4303 (ESP), RFC 6238 (TOTP).

You started from zero. Now you can read a packet, choose the right cipher, design a secure network, and plan for the quantum future.

Keep practicing, stay ethical, and keep learning.